Kernels Pequenos em CNNs para Aceleração Winograd: Quanto Custam e Como Recuperar a Acurácia

Rafael Silva de Souza
Curso de Engenharia de Computação
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
rafael.souza@inf.ufrgs.br

Mateus Grellert da Silva
Professor Adjunto, Orientador
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
mateus.grellert@inf.ufrgs.br

Resumo*—O algoritmo de Winograd faz convoluções com menos multiplicações, mas só compensa com kernels pequenos, como 2×2 e 3×3, enquanto redes clássicas, como a AlexNet, usam kernels bem maiores. Este artigo mede quanto custa trocar esses kernels e como recuperar a acurácia perdida. Para isso, fizemos 288 treinos no Tiny ImageNet e no CIFAR-100, quase todos com uma única receita e mudando um fator de cada vez, e avaliamos cada rede em FP32 e em INT8. O custo depende da rede e do classificador. Com classificador GAP e BatchNorm, passar de 3×3 para 2×2 custa de 1,0 a 2,4 pontos percentuais (p.p.) na AlexNet, na VGG16 e na ResNet-18. Na AlexNet com GAP, essa perda some quando as duas redes têm a mesma capacidade, e a rede inteira em 2×2 fica só 0,5 p.p. abaixo dos kernels originais, com 69% menos multiplicações; com classificador totalmente conectado, a mesma AlexNet perde 6,5 p.p. (uma seed). O stride da primeira camada pesa mais que o kernel (mediana de +9,7 p.p.), duas convoluções pequenas por estágio recuperam a perda, e a quantização para INT8 quase não muda a acurácia. Com isso, uma AlexNet só com kernels 2×2 chega a 60,63% em INT8, contra 59,45% da VGG16 (médias de 3 seeds), com um sexto dos parâmetros. Por fim, simulamos a aritmética de um acelerador Winograd em FPGA em 16 redes: o F(2,3) custa 0,2 p.p. (mediana), o F(4,3) custa 6,3 p.p., e o F(6,3) derruba seis das redes profundas para perto do acaso.

I. Introdução

O algoritmo de Winograd faz a mesma convolução com menos multiplicações [1], mas é usado quase sempre com kernels pequenos, porque a precisão numérica piora rápido com o tamanho da convolução [2]. Redes clássicas seguem outro caminho: a AlexNet [3], por exemplo, começa com kernels 11×11 e 5×5. A pergunta natural é quanto se perde ao trocar esses kernels por outros menores.

A literatura não dá uma resposta única. De um lado, a VGG mostrou que duas convoluções 3×3 seguidas enxergam a mesma região que uma 5×5 [4]. De outro, trabalhos mais recentes obtêm bons resultados com kernels grandes, usando convoluções *depthwise* [5], [6], um tipo de convolução que não recebe a aceleração de Winograd nas implementações que medimos [7].

Em um relatório anterior [7], comparamos variantes da AlexNet que diferiam em mais de um aspecto e que treinaram por números diferentes de épocas. Isso atrapalha a leitura dos resultados, porque o treino pode pesar mais que a própria arquitetura [8], [9] e uma comparação justa precisa controlar o que muda [10]. Neste artigo, refazemos o estudo com uma única receita de treino, mudando uma coisa de cada vez. A Seção II explica os conceitos usados, e a Seção III descreve o método e o motivo de cada escolha. A Figura 1 resume o caminho da AlexNet original até a melhor rede, e os resultados respondem a dez perguntas:

1) De onde vem a baixa acurácia da AlexNet em imagens pequenas? (Seção IV-A)
2) Quanto custa restringir o kernel? (Seção IV-B)
3) O 2×2 perde pelo kernel ou pela capacidade? (Seção IV-C)
4) Dá para recuperar a perda sem um corpo de kernel grande? (Seção IV-D)
5) A quantização INT8 atrapalha? (Seção IV-E)
6) Quanto da rede cabe em um acelerador Winograd? (Seção IV-F)
7) Os resultados valem em outras condições? (Seção IV-G)
8) Os ganhos se somam? (Seção IV-H)
9) Blocos Bottleneck e Fire barateiam o melhor corpo? (Seção IV-I)
10) Quanto a aritmética do acelerador Winograd custa em acurácia? (Seção IV-J)

Nossas principais conclusões são:

- O custo do kernel depende da rede e do classificador. Com GAP e BatchNorm, passar de 3×3 para 2×2 custa de 1,0 a 2,4 p.p. nas três famílias testadas. Com FC e Dropout, esse custo sobe na AlexNet e some na VGG16.
- Na AlexNet com GAP e BatchNorm, o 2×2 perde para o 3×3 por ter menos capacidade, não por causa do kernel: com os mesmos MACs e parâmetros, os dois empatam.
- O stride da conv1 (+9,7 p.p.) e o BatchNorm (+5,2 p.p.) pesam mais que trocar os kernels originais por 3×3 (−1,5 p.p.) ou 3×3 por 2×2 (−2,5 p.p.), nas medianas de todos os pares, com e sem FC.
- Duas convoluções pequenas por estágio recuperam a perda. Com esse corpo, o 2×2 fica acima do 3×3 nos

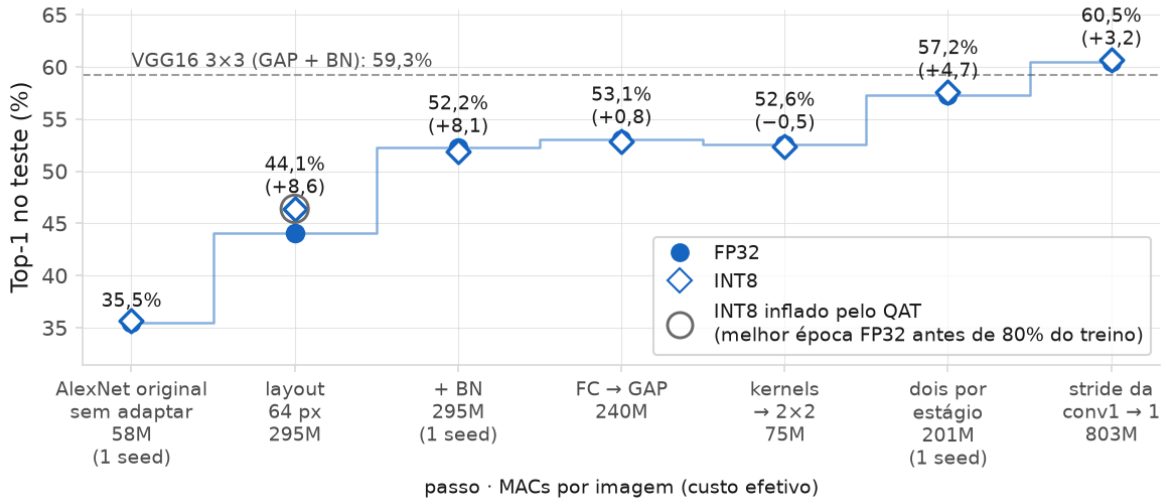


Figura 1. Da AlexNet original à melhor rede, mudando um fator por vez (top-1 no teste; média das seeds quando há mais de uma). Os dois últimos passos têm dez convoluções; os anteriores, cinco. Em cima de cada passo, a acurácia FP32 e, entre parênteses, quanto ele ganhou. Embaixo, os MACs em custo efetivo. Os maiores passos são o layout, o BN, as duas convoluções por estágio e o stride 1; trocar todos os kernels por 2×2 custa 0,5 p.p. e corta 69% dos MACs. A linha tracejada é a VGG16 3×3 com GAP e BN.

8 pares da seed 42, com cerca de metade dos MACs. Com as seeds 43 e 44, a vantagem se repete nas três seeds sem atalho e em duas de três com atalho.

- A quantização INT8 quase não muda a acurácia, seja qual for o kernel. Os grandes "ganhos" em INT8 vêm de treino extra.
- Na aritmética do acelerador Winograd, o tamanho do bloco de saída custa mais acurácia do que a troca de kernel custou no estudo principal: F(2,3) quase não custa, F(4,3) custa cerca de 6 p.p., e F(6,3) leva seis redes profundas ao acaso.
- Boa parte dos ganhos atribuídos antes a blocos de compensação vinha das diferenças de treino.

II. Fundamentos

A. Convolução de Winograd

Uma convolução comum desliza o kernel sobre o mapa de entrada e multiplica cada peso por cada valor coberto. O algoritmo de Winograd, em vez disso, calcula um bloco inteiro da saída de uma vez: transforma um trecho da entrada e o kernel para outro domínio, multiplica elemento a elemento e transforma o resultado de volta [1].

Escrevemos F(m,r) para blocos de saída $m \times m$ e kernel $r \times r$. O método direto usa $m^2 r^2$ multiplicações, e o de Winograd, $(m + r - 1)^2$. Assim, F(2,2) usa 9 em vez de 16 (1,78 vez menos), F(2,3) usa 16 em vez de 36 (2,25 vezes menos) e F(4,3) usa 36 em vez de 144 (4 vezes menos).

Três consequências importam neste artigo. A primeira é que **o ganho tem um preço numérico**: o limite do erro cresce exponencialmente com o tamanho da convolução [2], e por isso se usam kernels e blocos pequenos. A segunda é que **um kernel menor economiza menos**: o 2×2 ganha menos com Winograd que o 3×3, e o 1×1 não ganha nada, pois a fórmula dá m^2 nos dois métodos. A terceira é que **nem toda convolução se beneficia**. Consideramos elegíveis só as convoluções densas de stride 1, porque o stride maior que 1 quebra os blocos de saída vizinhos e as convoluções *depthwise* não recebem a aceleração nas implementações que medimos [7].

B. Geometria da rede

A AlexNet tem cinco convoluções: chamamos a primeira de *conv1* e as demais de *corpo*. Três escolhas definem sua geometria: o *kernel* (a região que cada convolução olha), o *stride* (quantos pixels ela avança a cada passo) e os *max-pools* (que reduzem o mapa pela metade). Juntas, elas determinam o *mapa final*, a grade de valores que chega ao classificador. Já o *campo receptivo* é a região da imagem que influencia uma unidade da rede; duas convoluções 3×3 seguidas, por exemplo, têm o campo receptivo de uma 5×5, com menos pesos [4].

C. Classificador e regularização

O *classificador* é a parte final da rede, e usamos dois tipos. No *GAP*, tiramos a média de cada canal do mapa final e usamos uma única camada linear. No *FC*, que tem muito mais parâmetros, o mapa é redimensionado para 6×6 (na VGG16, 7×7) e passa por três camadas totalmente conectadas, como na AlexNet e na VGG originais. Duas técnicas de regularização aparecem ao longo do texto: o *BatchNorm* (BN), que normaliza as ativações de cada camada, e o *Dropout*, que desliga metade das unidades, ao acaso, durante o treino.

D. Quantização

Em *FP32*, a rede usa números de ponto flutuante de 32 bits; em *INT8*, inteiros de 8 bits, que ocupam um quarto da memória e são mais baratos em hardware. O *Quantization-Aware Training* (QAT) é um treino que simula os arredonda-

mentos do INT8, para que a rede se ajuste a eles antes da conversão [24].

E. Medidas de custo

Medimos o custo em *MACs*, as multiplicações por imagem pelo método direto, e em *parâmetros*, os pesos da rede. Preferimos os MACs à latência porque eles não dependem da máquina. Eles não incluem a economia de Winograd, que estimamos à parte na Seção IV-F.

Algumas comparações usam o *custo efetivo*. Quando o mapa final é menor que 6×6, o classificador FC recebe cópias repetidas dos mesmos valores, e os pesos que só multiplicam essas cópias não acrescentam nada à função da rede. A AlexNet sem adaptação em 64 px, por exemplo, tem 57,8M de parâmetros, mas só 21M fazem diferença. O custo efetivo desconta esses pesos e é usado nas variações de custo da Figura 2 e nas Figuras 1, 9 e 10.

III. Metodologia

A. Visão geral

Cada rede passa pelas mesmas quatro etapas:

1) Treino em FP32 por 500 épocas, do qual guardamos a melhor época segundo o conjunto de validação.
2) QAT por 50 épocas, partindo dessa melhor época.
3) Conversão da melhor época do QAT para INT8, que é avaliado em CPU.
4) Avaliação de FP32, QAT e INT8 no conjunto de teste. Salvamos também as saídas da rede, para responder a perguntas futuras sem treinar de novo.

Dois princípios guiam o estudo. O primeiro é usar **uma única receita**, porque comparar arquiteturas exige um treino fixo [10] e uma receita diferente pode mudar a ordem entre redes [8], [9]. O segundo é **mudar uma coisa de cada vez**, para que cada diferença tenha uma única causa possível.

B. Dados

O conjunto principal é o Tiny ImageNet [11], com imagens 64×64 de 200 classes, 100 mil imagens de treino e 10 mil de validação oficial. Escolhemos esse conjunto pela velocidade, já que 288 treinos longos no ImageNet completo seriam inviáveis, e porque conclusões sobre hiperparâmetros obtidas em versões reduzidas do ImageNet costumam valer para o conjunto completo [12].

Usamos 90% do conjunto de treino para treinar e 10% para escolher a melhor época [13], e os resultados deste artigo vêm sempre da validação oficial, usada como teste. A separação é necessária porque quem escolhe o modelo não pode ser quem o mede: um número medido no mesmo conjunto que escolheu a época sairia otimista [14]. A divisão 90/10 muda com a seed, mas o teste é o mesmo para todas. Em todos os casos, reportamos a acurácia top-1.

O segundo conjunto é o CIFAR-100 [15], com imagens 32×32 de 100 classes, usado para ver se a ordem dos kernels depende dos dados. Nesses treinos, só os dados mudam: até a política do AutoAugment continua a mesma, porque trocá-la seria uma segunda mudança.

C. Receita de treino

A Tabela I traz cada escolha e o motivo. Cada valor vem de uma referência, exceto o número de épocas, que é escolha nossa e está declarado como tal.

Essa receita substituiu uma anterior, baseada em AdamW, com escolhas sem referência. Todos os treinos foram refeitos com a nova receita, e os antigos foram arquivados. Para garantir que ela seja seguida, testes automáticos falham se alguma configuração sair dela; as únicas exceções são os testes de robustez, em que cada configuração declara a mudança que faz.

A receita não é a melhor para cada rede: a ResNet e a MobileNetV2, por exemplo, têm receitas próprias. Aceitamos essa perda de propósito, em troca de poder comparar as redes entre si.

D. Redes

Cada família de redes entra no estudo por um motivo. A AlexNet é a rede principal, porque tem os maiores kernels entre as clássicas e é simples o bastante para mudar um fator sem mexer no resto. A VGG16 já nasceu com 3×3 e mostra o efeito do 2×2 em uma rede profunda, enquanto a ResNet-18 permite separar a primeira camada (7×7) dos blocos (3×3). Já a MobileNetV2 e as redes de compensação representam outra forma de economizar: Bottleneck e Fire [33] reduzem os canais com convoluções 1×1, e DepthwiseSep usa convoluções *depthwise*.

A AlexNet original foi feita para imagens 224×224 e, em 64×64, reduz a imagem rápido demais. Por isso, comparamos três layouts (Tabela II): o *original*, da rede sem adaptação; o de *64 px*, que reduz o stride da conv1 e os max-pools; e o de *stride 1*, que mantém a resolução por mais tempo. Os canais são sempre 64, 192, 384, 256 e 256.

Quanto aos kernels, partimos do conjunto *11-5-3-3-3* da AlexNet original e testamos também 3×3, 2×2 e 5×5 em todas as convoluções, o *alternado 3-2* (3-2-3-2-3) e a conv1 11×11 com corpo 3×3 ou 2×2. As convoluções de kernel ímpar usam o preenchimento que mantém o tamanho do mapa, de modo que só o kernel muda. O 2×2 exige um cuidado a mais: um kernel par não tem centro, e preencher de um lado só desloca o mapa em meio pixel por camada. Por isso, toda convolução 2×2 de stride 1 usa preenchimento simétrico [23]. Esse método ganha 2,5% no ImageNet, a mesma ordem do efeito de kernel que medimos, de modo que o preenchimento de um lado só inflaria o custo do 2×2.

O corpo também varia. Além do corpo de 5 convoluções, testamos *dois por estágio*, em que cada convolução vira duas seguidas (10 no total), e dois por estágio com *atalho* residual [13]. A ideia é que duas convoluções pequenas enxergam mais longe que uma, com uma não linearidade a mais [4].

Para separar o kernel da capacidade, usamos controles de largura: alargamos a rede 2×2 (canais ×1,5) até ela gastar o mesmo que a 3×3, e estreitamos a 3×3 (canais ×0,67) até gastar o mesmo que a 2×2. Os MACs ficam dentro de 2%, e os parâmetros também coincidem (2,33M contra 2,30M, e 1,04M contra 1,05M). Com o mesmo objetivo de isolar um fator, cada

Tabela I
RECEITA DE TREINO EM FP32: O QUE USAMOS E POR QUÊ

Escolha	Valor	Por quê
Otimizador	SGD, momentum 0,9, *weight decay* 5×10^{-4}	É a receita da AlexNet, e a VGG usa os mesmos valores [3], [4].
Taxa e lote	Taxa de aprendizado 0,01 e lotes de 128 imagens, em toda rede	São os valores da AlexNet [3]. Ajustar a taxa por rede faria o treino variar junto com a arquitetura.
Decaimento	Cosseno até zero, sem *warmup*	O cosseno funciona bem em arquiteturas variadas [16]. O *warmup* serve para taxas escaladas com lotes grandes, o que não é o caso aqui [17].
Perda	Entropia cruzada com *label smoothing* 0,1	É o outro refinamento que funciona em arquiteturas variadas [16], [18].
Duração	500 épocas fixas, sem parada antecipada	Com a taxa indo a zero no fim do orçamento, parar antes viraria mais uma variável [19]. O valor 500 é nosso: nenhum trabalho o fixa para o Tiny ImageNet. São 352 mil iterações, entre os treinos longos do CIFAR (78 a 234 mil) e os do ImageNet (cerca de 844 mil na AlexNet).
Aumento de dados	Deslocamento de até 4 pixels, espelhamento horizontal e AutoAugment	Esse é o deslocamento usado no ImageNet 64×64 [12]. Em proporção, ele equivale ao corte aleatório da AlexNet (32/256 = 8/64). O AutoAugment vem de [20].
Inicialização	He nas convoluções; normal com desvio 0,01 nas camadas lineares	Com desvio fixo nas convoluções, a escala do sinal cresceria com o tamanho do kernel, justamente a variável estudada. A inicialização de He compensa isso [21].
Classificador FC	Sempre com Dropout 0,5; sem Dropout só na comparação do próprio Dropout	É o classificador original da AlexNet e da VGG. Sem Dropout, uma rede chegou a 99,9% no treino e 38,5% na validação.
Precisão	Treino em precisão mista; avaliação em FP32	É a forma padrão de treino misto [22].

Tabela II
TAMANHO DO MAPA EM CADA CAMADA DA ALEXNET, COM ENTRADA 64×64

	Original	64 px	Stride 1
Stride da conv1	4	2	1
Kernel dos max-pools	3×3	2×2	2×2
Número de max-pools	3	2	2
Mapa depois da conv1	15×15	32×32	64×64
Depois do 1.º *max-pool* e da conv2	7×7	16×16	32×32
Depois do 2.º *max-pool* e da conv5	3×3	8×8	16×16
Depois do 3.º *max-pool*	1×1	—	—
Mapa final	**1×1**	**8×8**	**16×16**

rede pré-treinada na ImageNet tem uma gêmea treinada do zero, de modo que o pré-treino seja a única diferença.

E. Desenho dos experimentos

Os experimentos formam quatro estudos (fases 11 a 14 do projeto), resumidos na Tabela III.

O *estudo principal* mede cada fator isolado. Como uma grade completa teria 424 treinos e cerca de 3 mil horas de GPU, escolhemos os pares que respondem às perguntas e repetimos com outras seeds onde o efeito era pequeno.

Os *testes de robustez* perguntam se as conclusões dependem da receita ou dos dados. Cada um muda uma coisa declarada: a taxa de aprendizado (metade e dobro), o ajuste fino de redes pré-treinadas (100 épocas, taxas 10^{-3} e 10^{-4}, e QAT de 10 épocas com 1/100 da taxa) [32] ou o conjunto de dados.

O *estudo de combinações* pergunta se as técnicas que funcionaram se somam. Uma rede híbrida escolhida a dedo daria um número e não explicaria nada; por isso, ele é uma grade completa: kernel do corpo (2 opções), conv1 (2), corpo (3) e stride da conv1 (2) dão 24 redes, com a seed 42, das quais 18 são treinos novos e 6 vêm do estudo principal. Uma segunda etapa troca cada convolução do corpo de stride 1, sem atalho, por um bloco Bottleneck (1×1, $k\times k$ e 1×1) [13] ou Fire (uma 1×1 seguida de 1×1 e $k\times k$ em paralelo) [33], com um quarto dos canais de saída na redução e o kernel do corpo como único kernel espacial. Com um ou dois blocos por estágio e kernel 3×3 ou 2×2, são 8 redes; como a Fire 3×3 com um bloco por estágio é a própria Fire do estudo principal, são 7 treinos novos.

O *estudo da aritmética Winograd* mede quanto a conta do acelerador custa em acurácia. Para isso, usamos o simulador do acelerador Winograd em FPGA do grupo, que reproduz a aritmética de ponto fixo de F(2,3), F(4,3) e F(6,3) dentro do QAT. São 16 redes com kernels 3×3: 14 treinadas em FP32 com a mesma receita e 2 vindas do estudo principal. Cada rede FP32 passa depois por três QATs, um por tamanho de bloco, com as mesmas 50 épocas e a mesma taxa do QAT comum, de modo que os três braços de uma rede partem dos mesmos pesos e só diferem na aritmética. Comparamos cada braço com o FP32 da própria rede, no conjunto de teste, com uma seed.

F. Quantização

Chegamos ao INT8 por QAT porque, ao simular os arredondamentos durante o treino, ele dá à rede a chance de se ajustar a eles [24], [26]. Seguimos a regra usual de ajuste fino: 50 épocas (1/10 do FP32) e taxa de 10^{-4} (1/100 do FP32) caindo em cosseno até 10^{-6}, com o mesmo SGD [25]. Durante o QAT, o BN continua ativo, porque congelá-lo custou cerca de 1,5 p.p. em nossos testes e a literatura o mantém livre [25], [26], e as faixas de valores das ativações são atualizadas do início ao fim [24].

O formato é a definição padrão de INT8. As ativações têm 8 bits, com uma escala por tensor [24], e os pesos têm 8 bits, com uma escala por canal, simétricos entre -127 e 127 [25]. A última camada recebe e usa valores em INT8, mas sua saída fica em FP32, porque nenhuma camada quantizada a lê [25];

Tabela III
TREINOS POR ESTUDO (TOTAL: 288). ENTRE PARÊNTESES, OS TREINOS EXTRAS COM AS SEEDS 43 E 44

Bloco	**Treinos**
Estudo principal (152)	
AlexNet: kernel, classificador e BN	20 (+16)
AlexNet: stride da conv1 e max-pools	12
AlexNet: conv1 11×11 com corpo pequeno	4 (+8)
AlexNet: dois por estágio	4
AlexNet: controles de largura e 5×5	4 (+6)
AlexNet: Dropout e pré-treino	6
AlexNet: imagem ampliada para 224 px	5
VGG16: kernel e classificador	9 (+8)
VGG16: um kernel por bloco	9 (+10)
ResNet-18: kernel da conv1 e dos blocos	8 (+4)
MobileNetV2 e redes de compensação	19
Testes de robustez (39)	
Taxa de aprendizado 0,005 e 0,02	6
Ajuste fino de redes pré-treinadas	12
CIFAR-100 (3 seeds)	21
Estudo de combinações (35; mais 7 redes do principal)	
Kernel, conv1, corpo e stride da conv1	18 (+8)
Blocos Bottleneck e Fire no corpo de stride 1	7 (+2)
Aritmética Winograd (62)	
Redes FP32 novas	14
QAT com F(2,3), F(4,3) e F(6,3), 16 redes	48

quando essa saída também tinha 8 bits, a maior pontuação empatava em 14% a 31% das imagens.

A acurácia INT8 é medida com a biblioteca qnnpack, que soma em 32 bits sem erro. As bibliotecas onednn e fbgemm saturam a soma em 16 bits em CPUs sem VNNI, o que nos custou de 1 a 5 p.p. antes de identificarmos o problema. Como a acurácia não pode depender da CPU, antes de cada avaliação conferimos cada tipo de camada contra a conta exata em 32 bits. Além disso, o treino termina com erro se a concordância entre as respostas do QAT e do INT8 ficar abaixo de 0,90. Chamamos esse limite de *guarda*: ele não é uma métrica do artigo, e sim uma verificação de que a conversão funcionou.

G. Medidas

Além da acurácia e do custo, usamos três medidas. A *parcela Winograd* é a soma dos MACs das convoluções com stride 1, sem dilatação, sem grupos e com kernel 2×2 ou 3×3, dividida pelo custo efetivo da rede; a conta usa só a geometria das camadas. O *campo receptivo efetivo* mede quanto cada pixel de entrada move a unidade central da última convolução, na média de 200 imagens de teste [29], e dele reportamos a menor região que concentra metade dessa influência, em porcentagem da imagem. Por fim, as *imagens trocadas* vêm de comparar, para cada imagem de teste, se o FP32 e o INT8 acertam, já que duas redes com a mesma acurácia podem errar imagens diferentes [30].

H. Como comparamos

Toda comparação "A → B" usa pares de redes iguais em tudo menos em um fator, e reportamos a diferença B − A em pontos percentuais (p.p.), de modo que valores negativos indicam que B perdeu. Resumimos os pares pela mediana,

Tabela IV
FAIXA DE VARIAÇÃO ENTRE SEEDS EM FP32 (P.P.)

Família	**Um treino**	**Médias de 3 seeds**
AlexNet com BN	±0,7	±0,4
AlexNet sem BN e VGG16	±0,8	±0,5
ResNet-18	±1,0	±0,6

porque um mesmo fator pode ter efeitos bem diferentes conforme o resto da rede.

Para a mediana, calculamos um intervalo de confiança (IC) de 95% por *bootstrap*, reamostrando os pares 10 mil vezes [27]. Esse IC mostra como o efeito varia com o resto da rede, não o erro de uma medição, e não é um teste formal, porque os pares vêm do mesmo desenho e não são independentes. Com poucos pares, ele vale pouco: com dois, mostramos só os dois valores, e com quatro ou seis, deve ser lido como indicação.

A *variação entre seeds* é a que vem só do acaso. Para medi-la, repetimos configurações importantes de cada família com as seeds 43 e 44 [28]; a maioria dos treinos usa só a seed 42. Se SD é o desvio-padrão entre seeds, a diferença de dois treinos tem desvio $\sqrt{2}\,\mathrm{SD}$, e a faixa de cerca de 95% é $\pm 2\sqrt{2}\,\mathrm{SD}$ (Tabela IV); quando comparamos médias de 3 seeds, a faixa é $\sqrt{3}$ vezes menor. Tratamos como empate as diferenças dentro da faixa. Como cada faixa vem de poucas redes, uma diferença de uma seed pouco acima dela ainda é só uma indicação.

Quanto à origem dos números, o texto usa a média de 3 seeds sempre que ela existe, enquanto a Figura 2 e as Tabelas VII e IX usam a seed 42 em todas as redes, para comparar na mesma base. Por isso, a mesma rede pode aparecer com dois valores próximos: a 3×3 tem 53,81% na média e 53,50% na seed 42.

Um grupo de treinos recebe tratamento à parte. Chamamos de *treinos marcados* aqueles cuja melhor época FP32 veio antes de 80% do total: neles, o QAT parte de pesos obtidos com a taxa de aprendizado ainda alta, e a mudança para INT8 inclui treino extra. O limiar separa bem os grupos, pois, no estudo principal, a melhor época vem até a 396 ou a partir da 427. As figuras marcam esses casos com um anel, hachura ou asterisco ("INT8 inflado pelo QAT"). Eles nunca são removidos dos gráficos, mas ficam fora das fronteiras de eficiência em INT8 e das medianas INT8 da Figura 2. As Figuras 19 e 20, no Apêndice, mostram as curvas de treino e de QAT.

I. Reprodutibilidade

Fixamos as seeds, usamos convoluções determinísticas e versões fixas das dependências. Cada treino registra a versão do código, a GPU, a CPU e uma assinatura do conjunto de dados. Testes automáticos passam todas as redes do estudo principal por QAT e INT8 e conferem o resultado contra a definição da literatura.

O treino FP32 das redes do estudo principal rodou em RTX 4090 (91), GTX 1080 Ti (35) e RTX 4060 Ti (26). Para o resultado não depender da máquina, desligamos o TF32, modo de GPUs recentes que troca precisão por velocidade. Medimos

Tabela V
ALEXNET DE LAYOUT 64 PX COM BN: ACURÁCIA FP32 (%) E MACS POR KERNEL

Kernels	GAP (3 seeds) FP32	GAP (3 seeds) MACs	FC + Dropout (seed 42) FP32	FC + Dropout (seed 42) MACs
5×5	49,25	464M	52,14	519M
11-5-3-3-3	53,07	240M	52,23	295M
3×3	53,81	167M	50,75	223M
Alternado 3-2	54,58	120M	50,00	175M
2×2	52,58	75M	45,68	130M

Tabela VI
VGG16 COM GAP E BN: 2×2 EM UM BLOCO POR VEZ (FP32, 3 SEEDS)

Onde está o 2×2	FP32 (%)	Diferença para 3×3 (p.p.)
Nenhum bloco (3×3)	59,27	—
Bloco 1	59,54	+0,27
Bloco 2	59,34	+0,07
Bloco 3	59,05	−0,22
Bloco 4	58,56	−0,71
Bloco 5	59,18	−0,09
Todos os blocos	56,84	−2,43

que a mesma seed, na mesma GPU, retreina uma rede GAP bit a bit; redes FC não têm essa garantia, porque uma operação do redimensionamento antes do classificador não é determinística em GPU, mas a faixa da Tabela IV já inclui essa variação.

IV. RESULTADOS

A. De onde vem a baixa acurácia inicial?

A baixa acurácia vem **mais da geometria da rede do que do kernel**. Para ver isso, mantivemos os kernels originais, o classificador FC + Dropout e a ausência de BN, e mudamos só o layout. A AlexNet sem adaptação, com um mapa final de 1×1, fica em 35,47%. Com o layout de 64 px, ela chega a 44,10% (3 seeds) e, com a imagem ampliada para 224 px, a 49,54%, mesmo sem nenhuma informação nova na imagem. São de 8,6 a 14 p.p. que não têm relação com o tamanho do kernel.

A Figura 15, no Apêndice, isola o stride: reduzi-lo de 4 para 2 ganha de 6,4 a 13,0 p.p. nos oito pares, com mediana de +9,7 p.p., mas triplica o custo (+202% de MACs). Por isso, daqui em diante, os dois lados de cada comparação de kernel usam sempre o mesmo layout.

B. Quanto custa restringir o kernel?

O custo **depende da rede e do classificador**. Para medi-lo, comparamos pares de redes iguais exceto pelos kernels; a Figura 2 mostra cada par e a mediana, ao lado dos outros fatores.

Trocar 11-5-3-3-3 por 3×3 custa 1,5 p.p. (IC de −3,3 a +0,7, que inclui o zero) e reduz os MACs em 25%, enquanto trocar 3×3 por 2×2 custa 2,5 p.p. (IC de −3,7 a −1,1) e reduz os MACs em 54%. O alternado 3-2 empata com o 3×3 na mediana e usa 24% menos MACs. Cada um desses passos pesa menos que o stride, o BN, o Dropout ou o pré-treino, que mudam a acurácia em 4 a 10 p.p. Já a troca completa de 11-5-3-3-3 por 2×2 custa 4,7 p.p. e perde em todos os 9 pares, um efeito da mesma ordem desses fatores.

As medianas, porém, misturam situações diferentes. A Tabela V separa as duas colunas com BN da AlexNet, e a Figura 3 mostra as quatro.

Com GAP e BN, **os kernels pequenos quase não custam**: o 3×3 fica 0,74 p.p. acima do 11-5-3-3-3, e o alternado 3-2, 0,77 p.p. acima do 3×3. O 2×2 fica 1,23 p.p. abaixo do 3×3, mas só 0,49 p.p. abaixo dos kernels originais, com 69% menos MACs. Todas essas diferenças passam da faixa de ±0,4 p.p. para médias de 3 seeds.

Com FC, Dropout e BN, a ordem se inverte e passa a seguir o tamanho do kernel: o 2×2 perde 5,1 p.p. para o 3×3 e 6,5 p.p. para o 11-5-3-3-3. Sem BN, o FC + Dropout também penaliza o 2×2 (39,60%, contra 44,10% do 11-5-3-3-3, 3 seeds). Não investigamos por que o classificador FC favorece kernels maiores nessa rede.

A VGG16 se comporta ao contrário da AlexNet (Figura 4). Com GAP e BN, o 2×2 custa 2,43 p.p. (56,84% contra 59,27%, 3 seeds); com FC e Dropout, empata (59,17% contra 58,83%, 3 seeds); e, com FC sem Dropout, fica 1,12 p.p. abaixo (uma seed). Na ResNet-18, que usa GAP e BN, o 2×2 custa 1,01 p.p. em relação ao 3×3 (41,86% contra 42,87%, 3 seeds). Portanto, a conclusão "o FC agrava o custo do 2×2" **vale para a AlexNet, não em geral**.

C. O 2×2 perde pelo kernel ou pela capacidade?

Na AlexNet com GAP e BN, o 2×2 perde **pela capacidade**. Uma rede 2×2 tem menos pesos e faz menos multiplicações que uma 3×3 com os mesmos canais, e os controles de largura separam as duas coisas (Figura 5a). Com os mesmos MACs e parâmetros, as redes empatam: 53,83% contra 53,81% com 167M de MACs, e 52,58% contra 52,59% com 75M (3 seeds cada). Esse controle não foi feito com FC, nem na VGG16 ou na ResNet-18; a perda maior do 2×2 com FC na AlexNet, portanto, continua sem explicação.

No sentido oposto, **kernels maiores não compram acurácia com GAP**. Com 5×5 em todas as convoluções, a rede custa 2,8 vezes o 3×3 e chega a 49,25%, ou 4,56 p.p. abaixo dele (3 seeds, Figura 5b). Com GAP e BN, portanto, gastar mais só compensou como largura, não como kernel maior. Com FC + Dropout, o 5×5 chega a 52,14% (uma seed), perto dos 52,23% do 11-5-3-3-3, o que combina com o que a Tabela V já mostrou.

Resta saber em que parte da rede o 2×2 custa. Na VGG16 com GAP e BN, trocamos o kernel de um bloco por vez (Tabela VI; Figura 18, no Apêndice). Colocar 2×2 nos blocos 1, 2, 3 ou 5 fica dentro da faixa de ±0,5 p.p. para médias de 3 seeds. O bloco 4 é o único que sai dela: custa 0,71 p.p., um efeito pequeno e provável, já que fica pouco acima da faixa. A perda maior só aparece quando todos os blocos usam 2×2, ou seja, **nenhum bloco sozinho concentra o custo**.

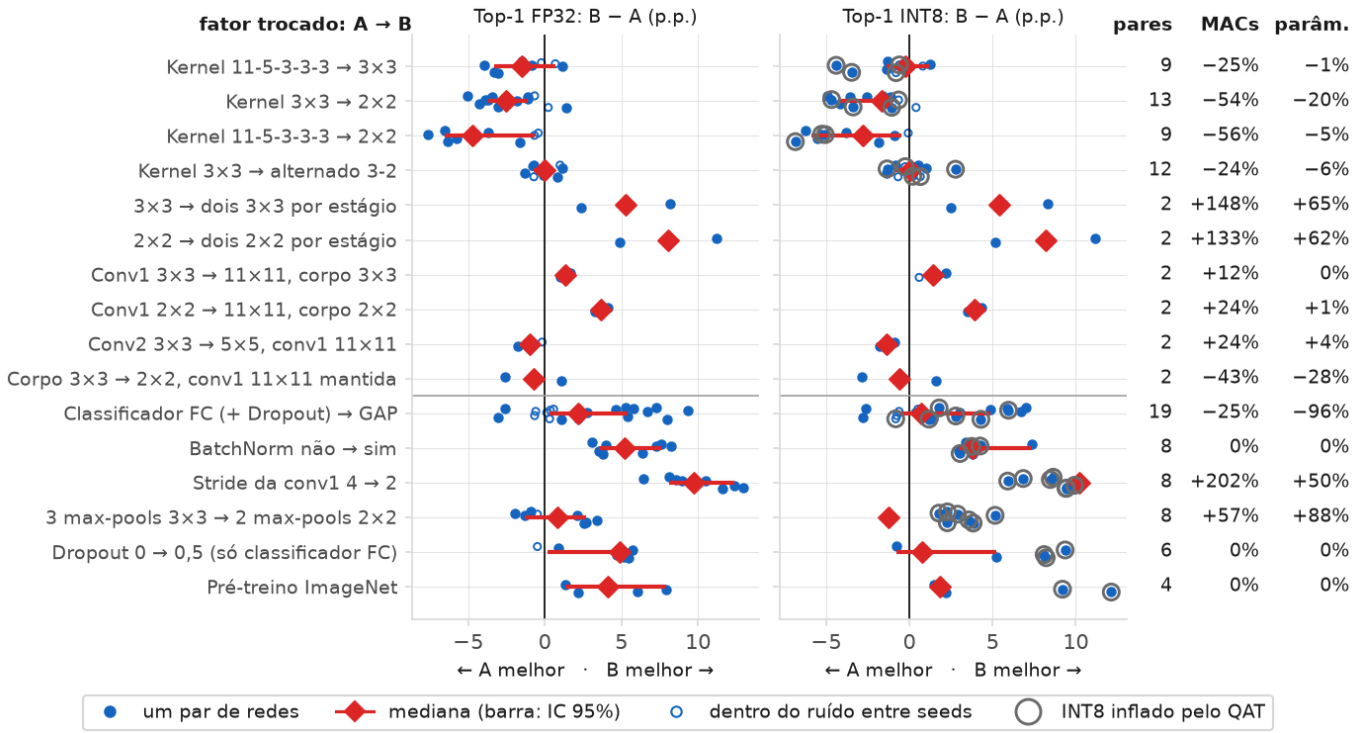


Figura 2. Efeito de trocar um fator de cada vez (seed 42). Cada ponto é um par de redes iguais no resto, com o valor B − A. O losango é a mediana dos pares, e a barra, seu IC 95%. Com dois pares não há barra: eles são a rede com GAP + BN e a com FC + Dropout + BN. No INT8, a mediana e o IC deixam de fora os pares com anel (INT8 inflado pelo QAT). Pontos vazados estão dentro da variação entre seeds. As dez primeiras linhas são de kernel. À direita, o número de pares e a variação mediana de MACs (custo efetivo) e de parâmetros.

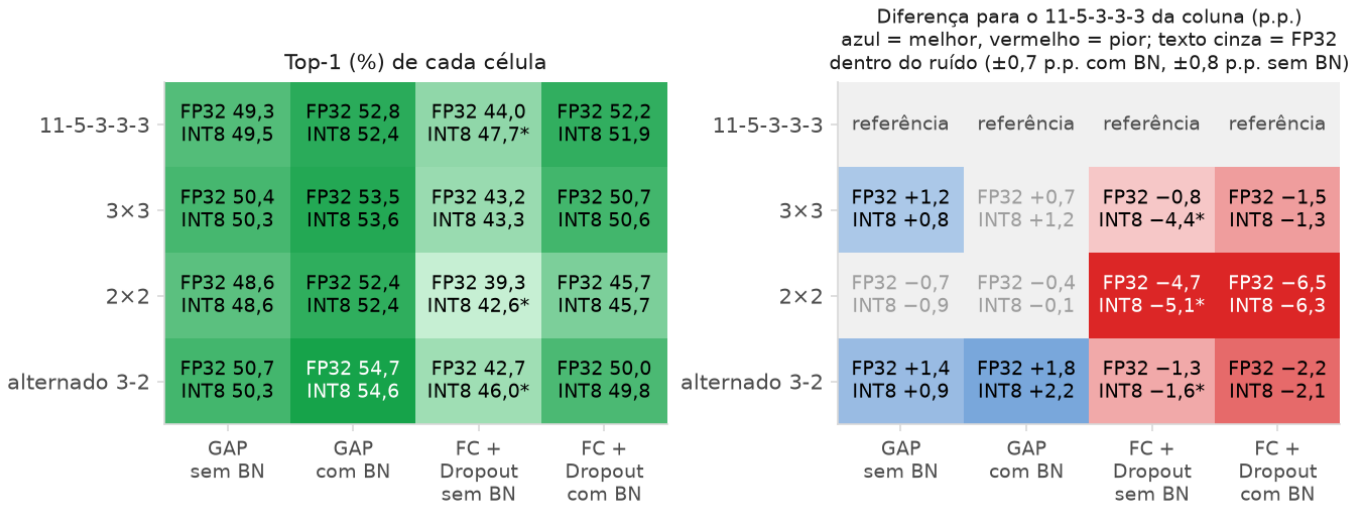


Figura 3. Kernel, classificador e BN na AlexNet de layout 64 px (seed 42). À esquerda, a acurácia de cada combinação; à direita, a diferença para o 11-5-3-3-3 da mesma coluna. Com GAP (duas primeiras colunas), os kernels pequenos ficam perto do original ou acima dele. Cinza: dentro da variação entre seeds; asterisco: INT8 inflado pelo QAT. Para comparar: a AlexNet sem adaptação fica em 35,5% em FP32, e com imagem ampliada para 224 px, em 49,5%.

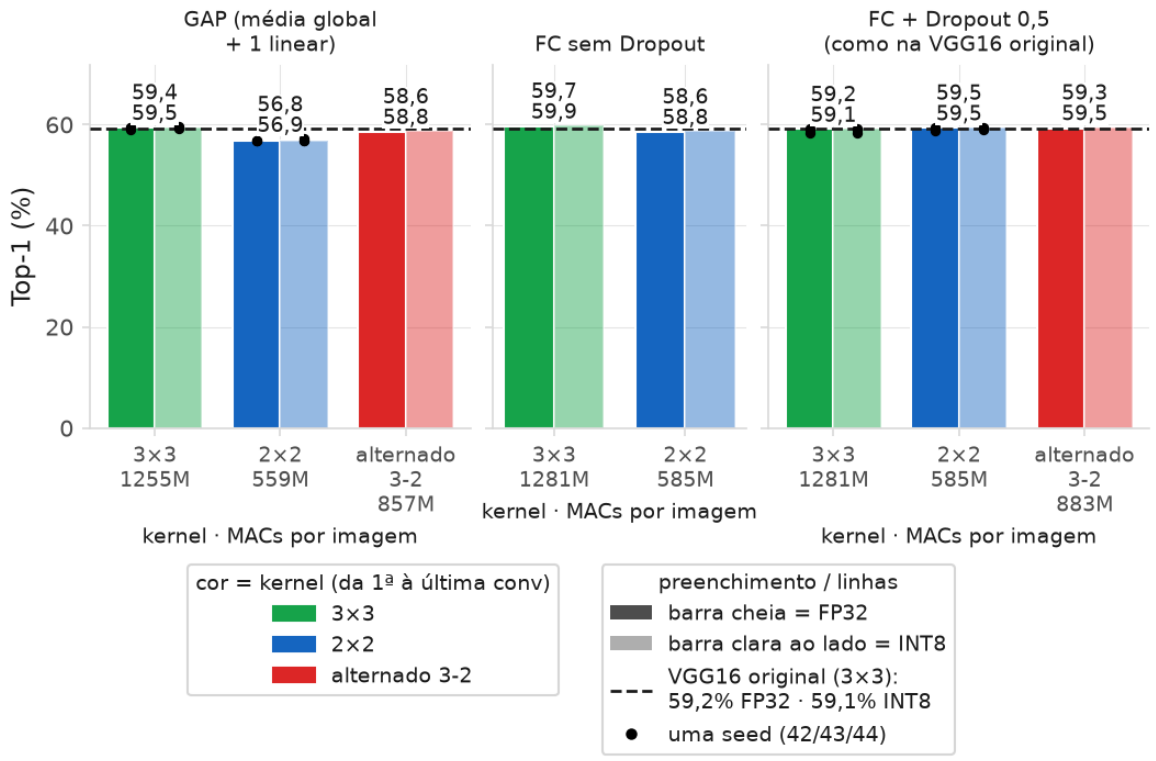


Figura 4. Kernel na VGG16 (13 convoluções, BN, do zero). Em cada painel, só o kernel das 13 convoluções muda. Barras: seed 42; pontos: cada seed, onde houver; linha tracejada: a VGG16 original (3×3, FC + Dropout). O 2×2 perde com GAP e, por menos, com FC sem Dropout (uma seed). Com FC + Dropout, empata.

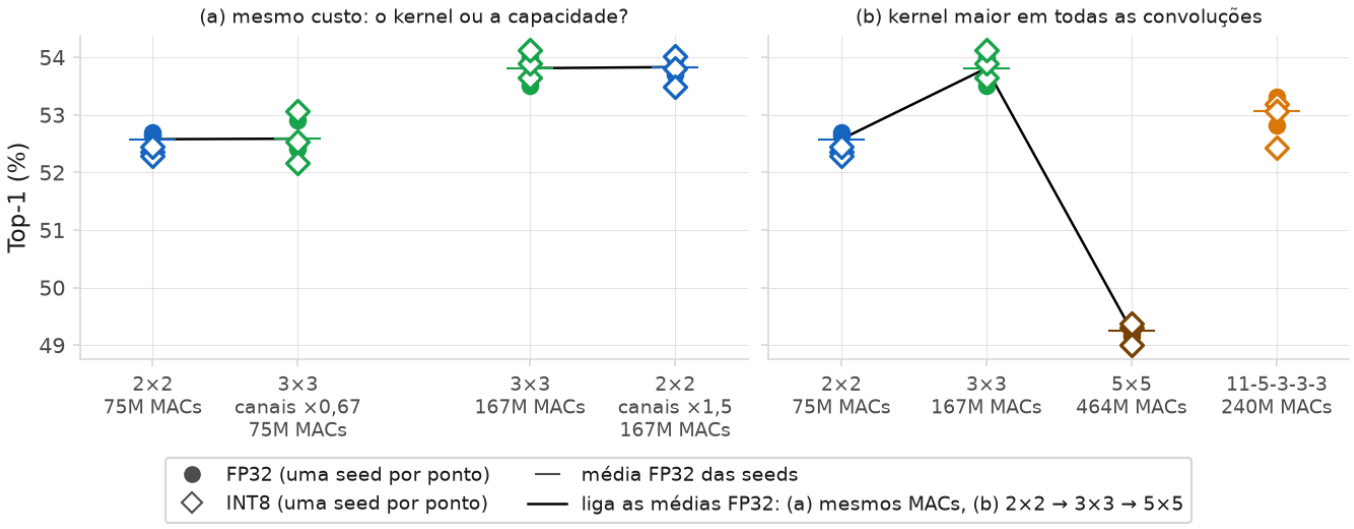


Figura 5. (a) Com o mesmo custo em MACs, 2×2 e 3×3 empatam. (b) Um kernel maior não ajuda: o 5×5 custa mais e fica bem abaixo do 3×3. Cada ponto é uma seed, com top-1 no teste. AlexNet de layout 64 px, GAP e BN.

D. Dá para recuperar a perda sem um corpo de kernel grande?

Sim, de duas formas: **mantendo um kernel grande só na conv1** ou **usando duas convoluções pequenas por estágio**. Testamos as duas na AlexNet de layout 64 px.

A primeira é a conv1 11×11 com corpo pequeno (Figura 6). Com corpo 2×2, essa rede chega a 55,93% em FP32 e 56,11% em INT8 (3 seeds), com 98M de MACs, e supera o 3×3 (53,81%, 167M) e os kernels originais (53,07%, 240M). Como a conv1 tem stride 2, ela já estaria fora de um acelerador de Winograd com qualquer kernel. Ainda assim, há um preço: com a conv1 maior, a parcela dos MACs que o acelerador pode executar cai de 98,4% para 75,2%.

A segunda forma são duas convoluções pequenas por estágio. Aqui o ganho depende do classificador, e por isso as medianas da Figura 2 (+5,3 e +8,0 p.p.) precisam ser lidas com cuidado, já que cada uma resume dois pares. Com GAP e BN, a configuração que recomendamos, o ganho é de +2,37 p.p. no 3×3 e +4,86 p.p. no 2×2 (seed 42, como na Figura 2); com FC + Dropout + BN, é de +8,15 e +11,21 p.p. Em troca, os MACs sobem 169% com GAP e BN, e 127% (3×3) e 97% (2×2) com FC. Com GAP e BN, a rede 2×2 com dois por estágio (57,25%, 201M) fica até acima da 3×3 equivalente (55,87%, 451M), com uma seed cada.

O campo receptivo efetivo dá uma pista de por que dois por estágio ajudam (Figura 7): a área com metade da influência vai de 17,8% para 29,8% da imagem no 2×2, e de 23,5% para 33,1% no 3×3. Já a conv1 11×11 quase não amplia essa área (18,5% contra 17,8%), embora melhore a acurácia. Seu ganho, portanto, não parece vir de enxergar mais da imagem, e não testamos a causa.

E. A quantização INT8 atrapalha?

A quantização **quase não altera a acurácia**, e o tamanho do kernel não muda esse quadro (Figura 8). Comparando o FP32 e o INT8 de cada rede no mesmo conjunto de teste, a diferença nos 126 treinos do estudo principal sem a marca de melhor época cedo vai de −0,38 a +0,58 p.p., com mediana de +0,04 p.p. No estudo de combinações, em que nenhum treino recebeu a marca, ela vai de −0,44 a +0,77 p.p.

Os ganhos grandes em INT8 aparecem apenas nos 26 treinos marcados, em que a diferença vai de +0,56 a +6,69 p.p., e **não são efeito da quantização**: ali o QAT continua treinando, agora com uma taxa de aprendizado baixa, uma rede que ainda não tinha terminado de convergir.

Acurácia igual, porém, não quer dizer respostas iguais [30]. A maioria das redes troca de 2% a 7% das imagens de teste entre certo e errado, em quantidades parecidas nas duas direções, enquanto as redes marcadas chegam, em geral, a 9–14% e quase sempre ganham no saldo (Figura 16, no Apêndice).

A exceção é a MobileNetV2 pré-treinada. Nos três treinos dela, a concordância entre QAT e INT8 ficou abaixo da guarda de 0,90 (0,859, 0,888 e 0,899); como esses treinos terminaram com erro depois de salvar os resultados, seus números não passaram na nossa verificação. Com ajuste fino, ela perde 1,49 p.p. do FP32 para o INT8 (taxa 10^{-4}) e 1,16 p.p. (taxa 10^{-3}, treino marcado), ao passo que, fora ela, nenhum treino sem marca dos dois primeiros estudos perde mais de 0,38 p.p. A MobileNetV2 treinada do zero passa na guarda (0,94) e quase não muda (47,83% para 47,88%). A rede pré-treinada é a única

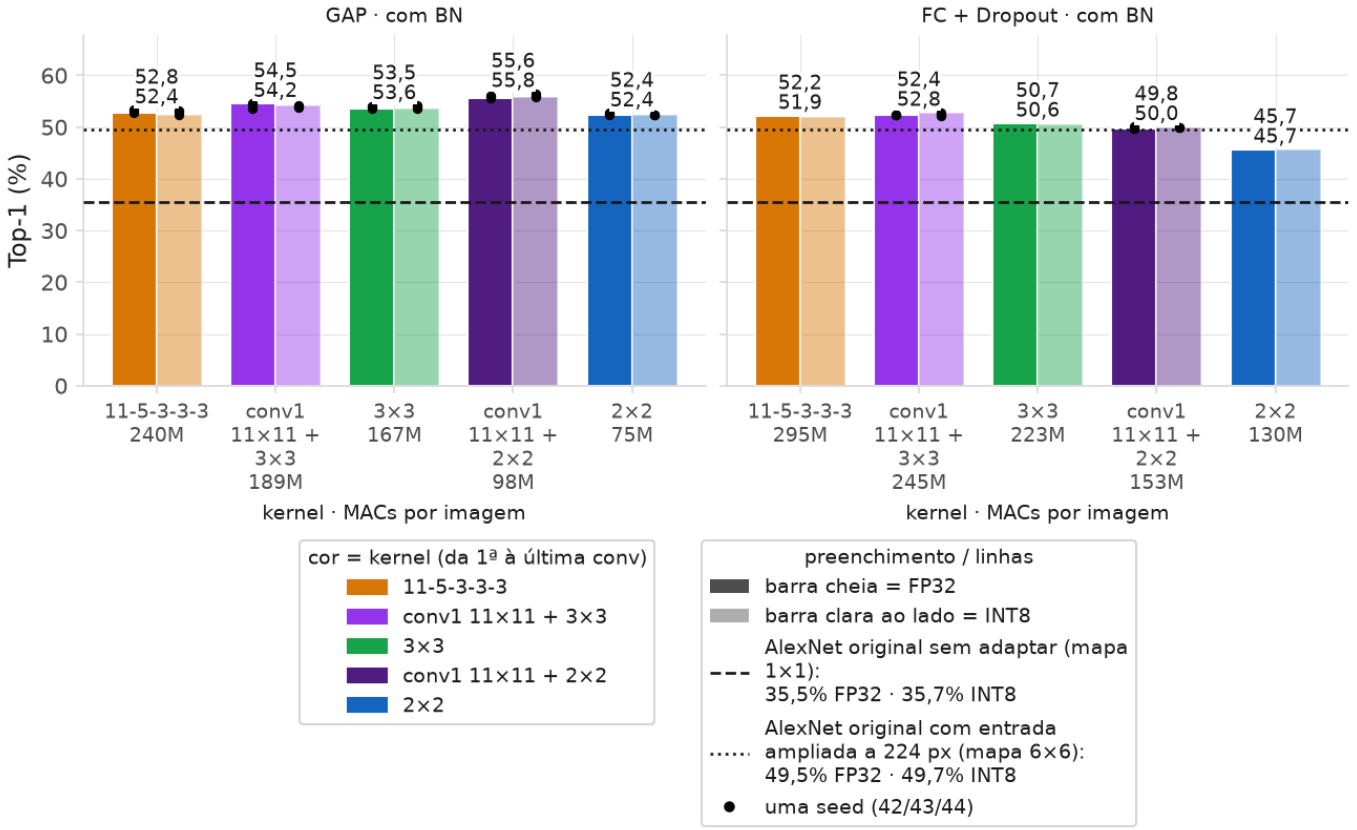


Figura 6. Conv1 grande com corpo de kernel pequeno, com BN, do zero. Com GAP (à esquerda), a conv1 11×11 com corpo 2×2 é a melhor das cinco e a segunda mais barata. Barras: seed 42 (cheia: FP32; clara: INT8), por isso diferem das médias do texto; pontos: cada seed. Linhas: a AlexNet sem adaptação (tracejada) e com a imagem ampliada para 224 px (pontilhada).

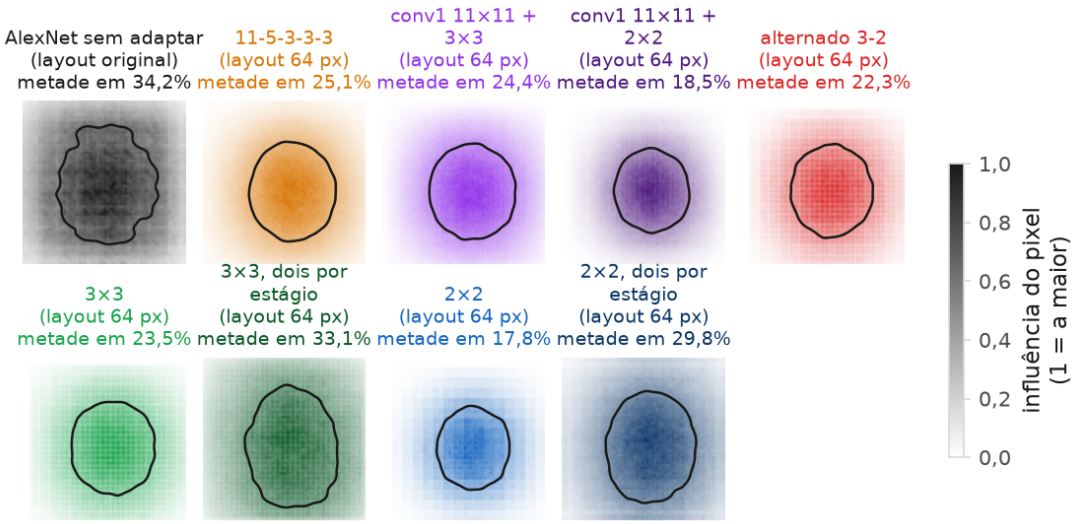


Figura 7. Que região da imagem cada rede usa: quanto cada pixel move a unidade central da última convolução (média de 200 imagens de teste, seed 42). O contorno marca a menor região com metade da influência. Todas têm GAP e BN e o layout de 64 px, menos a AlexNet original (FC + Dropout, sem BN, layout original). Compare, na segunda linha, cada kernel com sua versão de dois por estágio.

com pesos da ImageNet e convoluções *depthwise*, mas não investigamos a causa.

F. Quanto da rede cabe em um acelerador Winograd?

A resposta depende da família, e **uma rede barata não é necessariamente adequada ao acelerador**. As Figuras 9 e 10

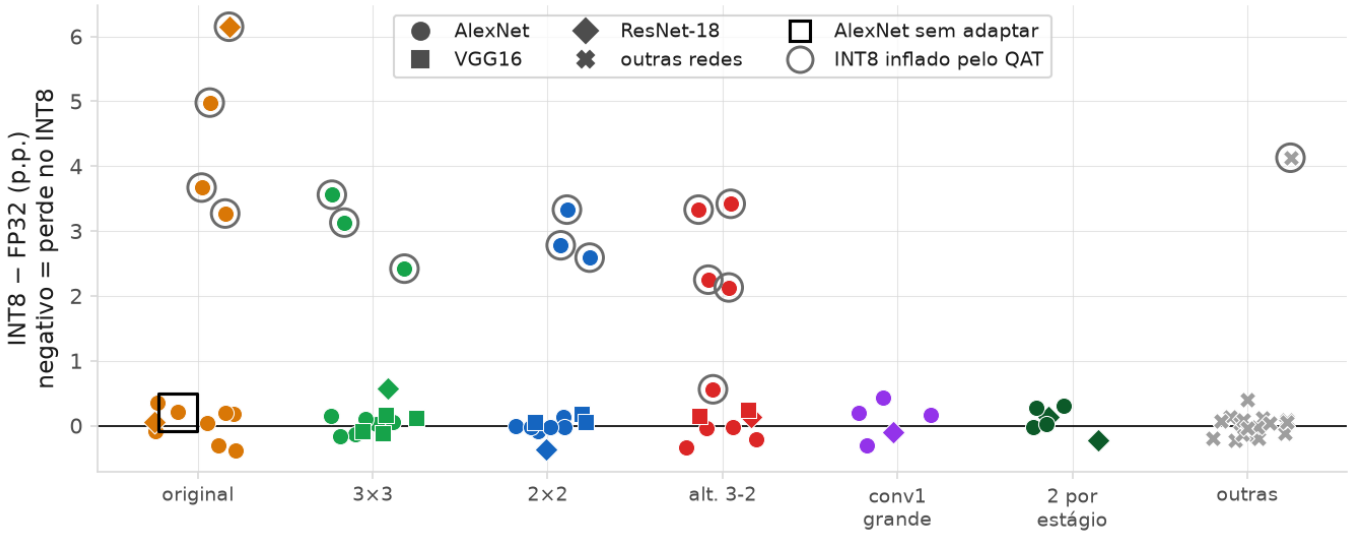


Figura 8. Quanto cada rede muda do FP32 para o INT8, por tipo de kernel (seed 42, top-1 no teste). Valores negativos indicam perda em INT8. Sem a marca do anel, todas as redes ficam perto de zero, seja qual for o kernel. Os pontos altos, com anel, são treino extra do QAT. O quadrado marca a AlexNet sem adaptação. Grupos: original = 11-5-3-3-3 ou 7-3 (ResNet-18); conv1 grande = conv1 11×11 ou 7×7 com corpo pequeno; 2 por estágio inclui a conv1 profunda da ResNet-18; outras = redes de compensação e MobileNetV2.

comparam acurácia e custo, e nelas a linha tracejada liga as redes que nenhuma outra supera nos dois critérios ao mesmo tempo. Na parte barata, ela passa por redes de compensação, como a Bottleneck; na intermediária, pela conv1 11×11 com corpo 2×2 e pela 2×2 com dois por estágio; e, na parte cara, pela VGG16. Isso vale para o estudo principal: as redes de stride 1 do estudo de combinações (Seção IV-H) passam a VGG16 com menos MACs (Tabela VII).

A Tabela VII mostra a parcela dos MACs que o acelerador pode executar. As AlexNets de kernel pequeno e a VGG16 ficam perto de 100%, enquanto a TinyMobileNetV2 e a MobileNetV2 ficam em torno de zero, e a FinalBottleneck-Residual, em 28%. Essas últimas economizam parâmetros ou MACs, mas gastam a maior parte do cálculo em convoluções 1×1, *depthwise* ou de stride maior que 1, que o acelerador não aproveita [7]. A busca por uma rede eficiente e a busca por uma rede para Winograd levam, portanto, a **escolhas diferentes**; os blocos Fire sobre o melhor corpo ficam no meio do caminho (Seção IV-I).

É preciso lembrar que os MACs contam o método direto. Com Winograd, a vantagem do 2×2 diminui, porque esse kernel economiza menos (Seção II). Fizemos uma estimativa com a fórmula $(m + r - 1)^2$, aplicada só às convoluções elegíveis: com F(2,3), a rede 3×3 com dois por estágio cai de 451M para cerca de 202M multiplicações e, com F(2,2), a rede 2×2 equivalente cai de 201M para cerca de 114M. A vantagem do 2×2 passa, assim, de 2,2 vezes para cerca de 1,8 vez. Note que, com F(2,3), a rede 3×3 faz praticamente as mesmas multiplicações (202M) que a 2×2 pelo método direto (201M). Essa conta é aritmética, não uma medição: ela ignora o custo das transformações e a precisão numérica, e supõe um acelerador com F(2,2).

Tabela VII
ACURÁCIA, CUSTO E PARCELA DOS MACS QUE O ACELERADOR PODE EXECUTAR (SEED 42)

Rede	FP32 (%)	INT8 (%)	Parâm. (M)	MACs (M)	Winograd (%)
2×2, 2/est., atalho, stride 1	60,75	60,46	2,52	862	92,9
2×2, 2/est., stride 1	60,35	60,79	2,33	803	99,6
VGG16 3×3	59,35	59,47	14,82	1255	99,8
TinyMobileNetV2	57,96	58,00	1,51	969	0,4
2×2, 2/est.	57,25	57,55	2,33	201	99,2
SmallKernel + BN	57,20	57,17	1,60	461	99,7
FinalBottleneckResidual	55,83	55,63	0,57	54	28,3
Conv1 11×11 + corpo 2×2	55,64	55,84	1,08	98	75,2
FireBypass	55,19	55,27	0,52	178	79,7
Fire	54,57	54,44	0,52	178	79,7
Bottleneck	53,71	53,57	0,39	40	38,8
3×3	53,50	53,65	2,30	167	98,7
11-5-3-3-3	52,81	52,43	2,52	240	57,1
2×2	52,39	52,36	1,05	75	98,4
MobileNetV2 (do zero)	47,83	47,88	2,48	26	0,0
DepthwiseSep	46,53	46,66	0,31	20	0,0

2/est. = dois por estágio. A coluna Winograd conta 2×2 e 3×3. Em um acelerador só com F(m,3), as redes 2×2 ficam perto de zero.

G. *Os resultados valem em outras condições?*

Em boa parte, sim, mas **só testamos redes com GAP e BN**: os testes de taxa de aprendizado e de CIFAR-100 usam essa configuração, e o custo maior do 2×2 com FC na AlexNet não foi repetido fora do cenário original. A Figura 11 resume os cenários.

O custo do kernel se repete. Trocar 3×3 por 2×2 custa de 0,7 a 2,4 p.p. em FP32 em todos os cenários e quase nada quando os MACs são iguais. Trocar os kernels originais por 3×3 empata ou melhora um pouco nos cenários com AlexNet; na ResNet-18, a mesma troca fica em −0,95 p.p., dentro da

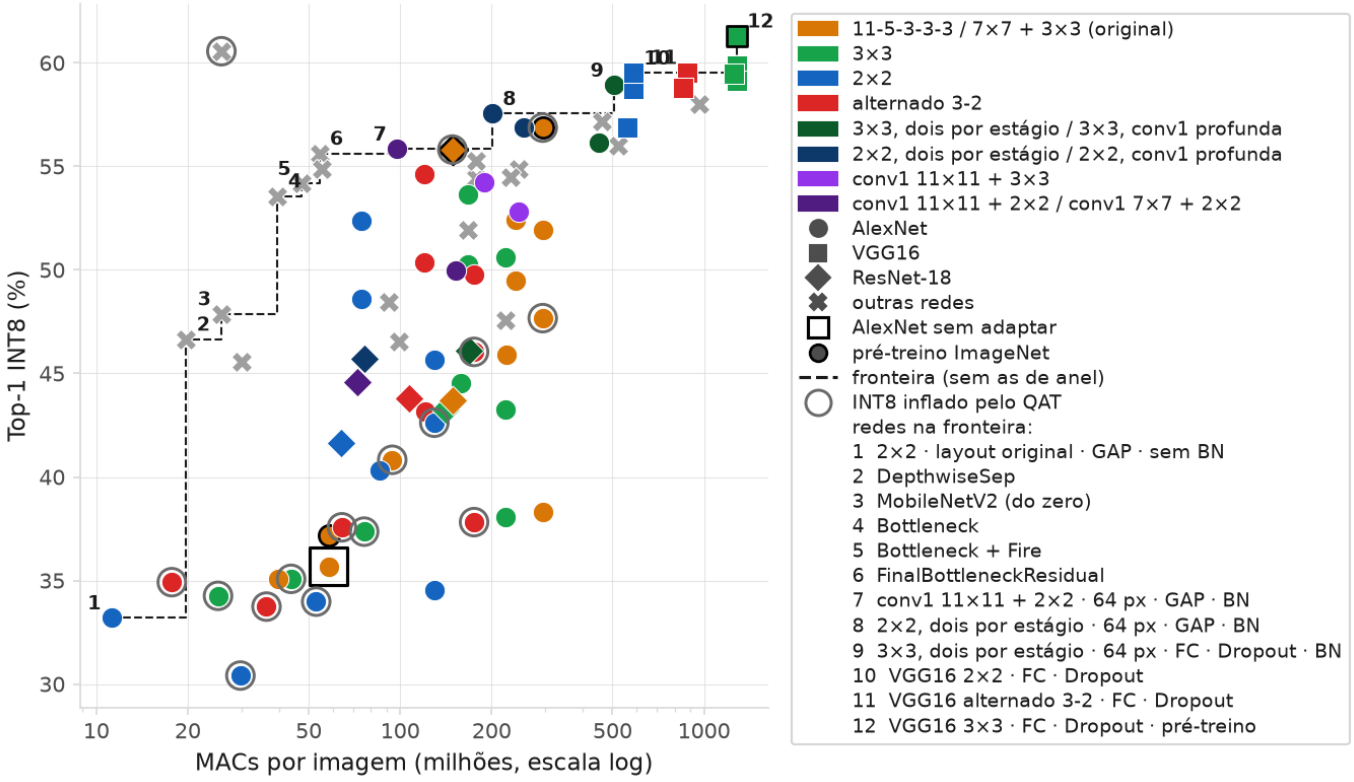


Figura 9. Acurácia INT8 e custo efetivo em MACs (estudo principal, seed 42). A linha tracejada liga as redes que nenhuma outra supera nos dois critérios ao mesmo tempo; as redes com anel ficam fora dela, porque seu INT8 está inflado pelo QAT. O quadrado marca a AlexNet sem adaptação.

Tabela VIII
ROBUSTEZ: ACURÁCIA FP32 (%) POR KERNEL

Cenário	11-5-3-3-3	3×3	Alt. 3-2	2×2
AlexNet, Tiny ImageNet, seed 42				
Taxa 0,005	52,15	52,27	—	51,57
Taxa 0,01 (receita)	52,81	53,50	—	52,39
Taxa 0,02	53,89	54,22	—	53,22
CIFAR-100, 3 seeds				
AlexNet	63,00	63,34	63,42	62,09
VGG16	—	71,08	70,99	70,31

Tabela IX
EFEITO DE CADA FATOR NAS 24 COMBINAÇÕES: MEDIANA DE B − A (SEED 42, P.P.)

A → B	FP32	INT8	B acima	Faixa FP32
Stride da conv1: 2 → 1	+3,03	+3,40	12 de 12	+1,43 a +4,16
5 convs → 2 por estágio	+2,12	+2,21	8 de 8	+0,87 a +4,86
Corpo 3×3 → 2×2	+1,23	+1,50	9 de 12	−1,28 a +1,82
2 por estágio → + atalho	+0,41	+0,16	6 de 8	−0,53 a +0,63
Conv1 → 11×11	−0,12	−0,19	4 de 12	−1,08 a +3,25

faixa de uma seed.

Com outras taxas de aprendizado (Tabela VIII), a diferença entre 3×3 e 11-5-3-3-3 fica dentro da faixa de uma seed nas taxas 0,005 e 0,02, então não afirmamos ordem entre eles. O 2×2 fica abaixo do 3×3 nas três taxas, de 0,7 a 1,1 p.p.; contra o 11-5-3-3-3, a diferença fica dentro da faixa nas taxas 0,005 e 0,02. Além disso, a taxa 0,02 fica acima da média das 3 seeds da taxa 0,01 nas três redes (+0,4 a +0,8 p.p., duas delas dentro da faixa), um sinal fraco de que a receita pode não ser a melhor para cada rede.

No CIFAR-100 (Tabela VIII; Figura 17, no Apêndice), a ordem dos kernels é a mesma do Tiny ImageNet nas duas famílias, com τ de Kendall igual a +1,00 [31]. Como são só quatro e três redes, e algumas ficam muito próximas, isso é um sinal de consistência, não uma prova.

Quanto ao pré-treino, a taxa de 0,01 da receita única apaga boa parte dele. Com ajuste fino de verdade [32], a taxa de 10^{-3} supera a de 10^{-4} em todas as redes testadas, e a VGG16 chega a 67,27%; o ganho sobre a mesma rede treinada do zero vai de +5,5 a +14,9 p.p. em 64 px. Esse resultado, porém, não responde à pergunta sobre kernels, porque os pesos pré-treinados só existem para os kernels originais de cada rede.

H. Os ganhos se somam?

Os ganhos se somam **apenas em parte**. Testamos todas as combinações na AlexNet com GAP e BN: corpo 3×3 ou 2×2; conv1 com o kernel do corpo ou 11×11; corpo de 5 convoluções, dois por estágio ou dois por estágio com atalho; stride da conv1 igual a 2 ou 1. São 24 redes, com a seed 42 (Figura 12, Tabela IX).

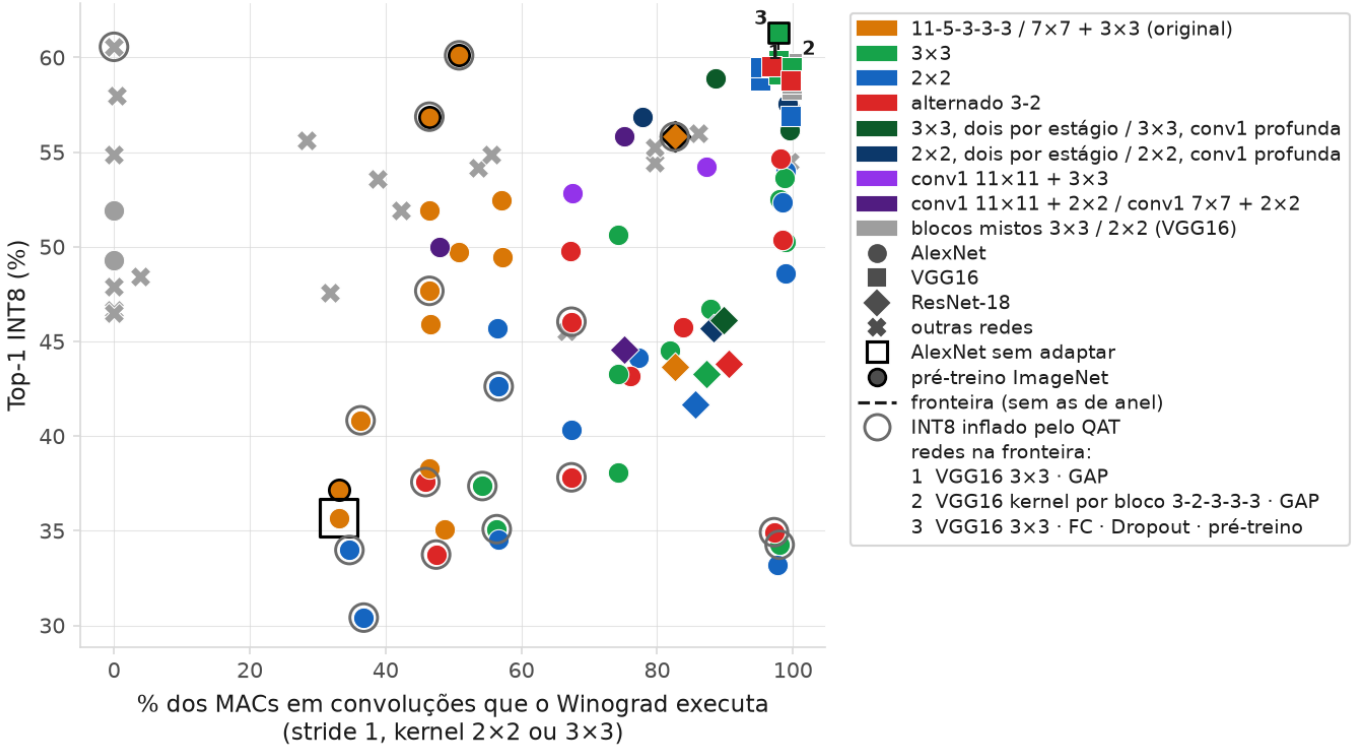


Figura 10. Acurácia INT8 e parcela dos MACs em convoluções que um acelerador Winograd pode executar (stride 1, kernel 2×2 ou 3×3, sem grupos; seed 42, custo efetivo). A linha tracejada liga as redes que nenhuma outra supera nos dois critérios (sem as de anel). Sem contar as redes com anel, as mais precisas, no canto superior direito, cabem quase inteiras. A conta usa só a geometria das camadas. Além das redes da Figura 9, entram as de imagem ampliada para 224 px e as VGG16 com um kernel por bloco (cinza).

Dois fatores nunca falham: o stride 1, que é o maior e custa quatro vezes mais MACs, e dois por estágio, que vêm em seguida. Já o atalho residual fica dentro da variação entre seeds, e seu sinal muda entre os pares, enquanto a conv1 11×11 só ajuda no corpo de 5 convoluções (3 de 4 pares): com dois por estágio, sua mediana é de −0,20 p.p., ou seja, a segunda convolução por estágio já entrega o que ela entregava.

O resultado mais curioso é o do kernel do corpo. Com 5 convoluções, a troca de 3×3 por 2×2 varia de −1,28 a +1,09 p.p. Com dois por estágio, o 2×2 vence nos 8 pares (de +0,47 a +1,82 p.p., mediana de +1,4), com cerca de metade dos MACs, e seis desses 8 pares passam da faixa de ±0,7 p.p. É um indício forte, mas de uma seed, e por isso repetimos os dois pares de stride 1 com as seeds 43 e 44. Sem atalho, o 2×2 vence nas três seeds (de +1,68 a +1,82 p.p. em FP32). Com atalho, vence em duas de três (+1,59, +1,48 e −0,18 p.p.): a 3×3 com atalho da seed 44 (60,10%) é a única dessas redes 3×3 acima de 59,2%, e sozinha fecha a diferença. A vantagem do 2×2 com dois por estágio é, portanto, **de cerca de 1 a 2 p.p., mas não é firme nos dois corpos**.

Os dois maiores ganhos, por sua vez, não se somam por inteiro. Partindo do 3×3 com stride 2 (53,50%), os ganhos separados de usar dois por estágio (+2,37) e stride 1 (+4,16) levariam a 60,03%, mas o valor medido é 58,53%. No 2×2, a soma levaria a 61,24%, e o valor medido é 60,35%: o corpo 2×2 perde menos nessa combinação.

As melhores redes usam **corpo 2×2, dois por estágio e stride 1**. Só com 2×2 e com atalho, a rede chega a 60,75% em FP32 e 60,46% em INT8 (2,52M de parâmetros, 862M de MACs); sem atalho, a 60,35% e 60,79% (2,33M, 803M). Com a conv1 11×11 e atalho, chega a 60,48% e 60,85%, o maior INT8 das 24, empatada com as outras duas. A VGG16 3×3 com GAP e BN fica em 59,35% e 59,47%, com 14,82M de parâmetros e 1255M de MACs (de 59,09% a 59,37% em FP32, em 3 seeds). As duas AlexNets só com 2×2 ficam, assim, de 1,0 a 1,4 p.p. acima dela, com cerca de um sexto dos parâmetros e dois terços dos MACs.

Como essas duas redes saíram do topo das 24, a seed 42 pode estar favorecendo as duas, e por isso as repetimos com as seeds 43 e 44. Nas médias de 3 seeds, a rede sem atalho fica em 60,47% (FP32) e 60,63% (INT8), e a com atalho em 60,35% e 60,34%, contra 59,27% e 59,45% da VGG16. A pior seed de cada uma (60,35% sem atalho e 59,92% com atalho, em FP32) fica acima da melhor seed da VGG16 (59,37%). O atalho continua sem efeito no corpo 2×2 (média de −0,12 p.p. em FP32, com o sinal trocando entre as seeds). No corpo 3×3, ele é positivo nas três seeds (média de +0,68 p.p.), mas vai de +0,05 a +1,37 p.p., e três seeds não bastam para medi-lo. Já as redes 3×3 com dois por estágio não superam a VGG16 com clareza: 58,70% sem atalho e 59,39% com atalho em FP32.

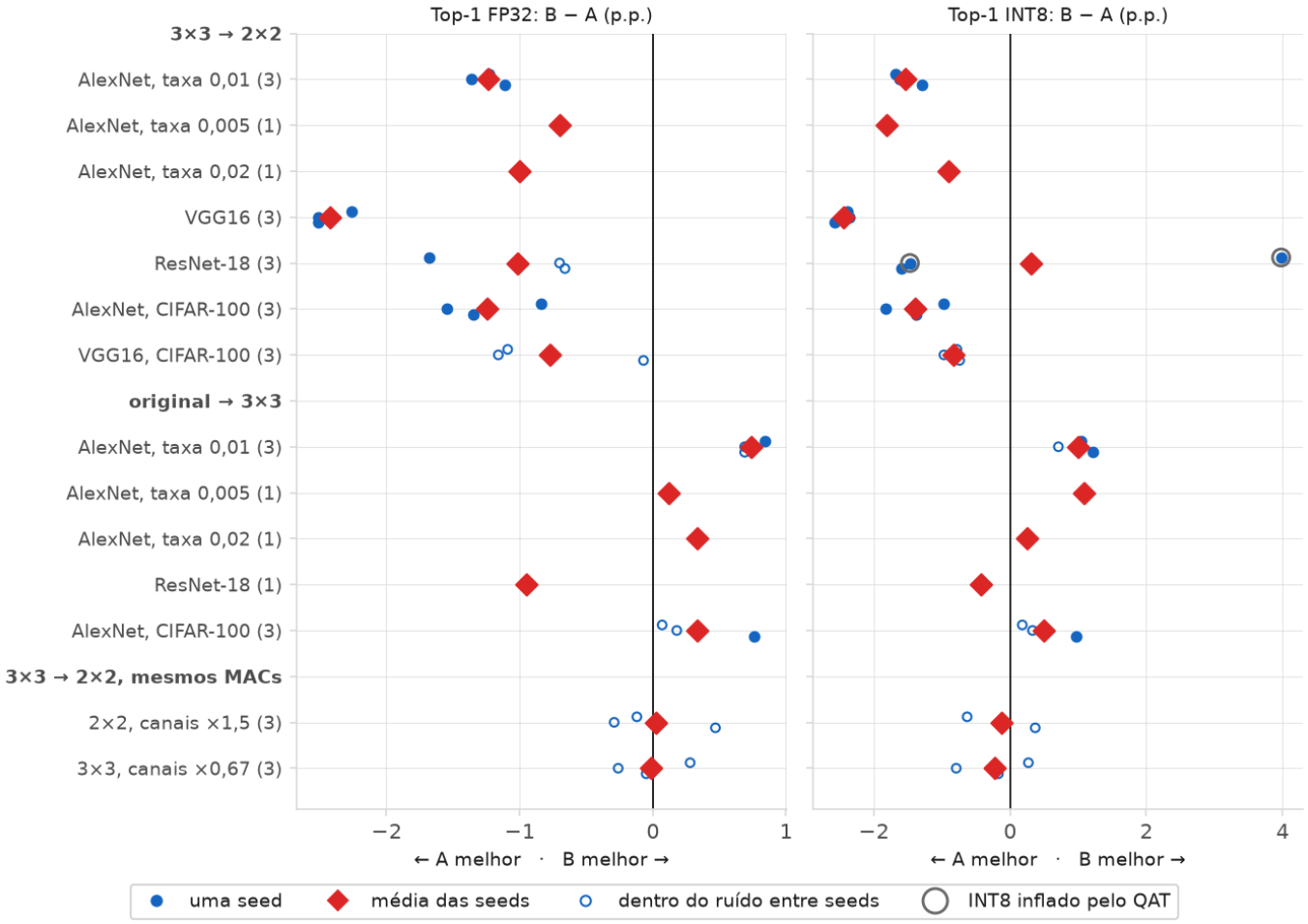


Figura 11. O efeito do kernel em cada cenário: duas redes iguais exceto pelo kernel (top-1 no teste de cada conjunto de dados: Tiny ImageNet, salvo onde diz CIFAR-100; entre parênteses, o número de seeds). O custo do 2×2 aparece em todas as linhas do primeiro grupo e some no último, em que os MACs são iguais (o 2×2 com 1,5 vez os canais, ou o 3×3 com 0,67 vez). O grupo do meio compara os kernels originais (11-5-3-3-3 na AlexNet, 7×7 na ResNet-18) com 3×3. Um ponto muito à direita no INT8 da ResNet-18, com anel, é um caso de INT8 inflado pelo QAT.

I. Blocos Bottleneck e Fire barateiam o melhor corpo?

Com dois blocos por estágio, o Fire barateia o corpo **a um custo pequeno ou nulo em acurácia**; com um bloco por estágio, nenhum dos dois consegue. Para chegar a essa resposta, partimos do corpo de stride 1 do estudo de combinações, sem atalho, e trocamos cada convolução por um bloco Bottleneck ou Fire com o mesmo kernel (Figura 13).

Com um bloco por estágio, os blocos cortam de 70 a 83% dos parâmetros e de 66 a 77% dos MACs, mas perdem de 1,68 a 3,56 p.p. em FP32 nas quatro trocas, sempre além da variação de ±0,7 p.p. Nesse caso, o Bottleneck fica à frente do Fire nos dois kernels (por 1,41 p.p. no 3×3 e 0,95 p.p. no 2×2).

Com dois blocos por estágio, a ordem se inverte: o Fire fica 1,27 p.p. (3×3) e 1,54 p.p. (2×2) acima do Bottleneck. No 2×2, o Fire fica um pouco abaixo do corpo comum nas três seeds (−0,24, −0,68 e −1,06 p.p. em FP32). Na média das três, são 59,81% contra 60,47% em FP32 (−0,66 p.p., fora da faixa de ±0,4 p.p. para médias de 3 seeds) e 59,57% contra 60,63% em INT8 (−1,06 p.p.), com um quarto dos parâmetros e dos MACs. No 3×3, ele fica 1,67 p.p. acima do corpo comum (60,20% contra 58,53%) e 1,50 p.p. acima da média de 3 seeds dele (58,70%), com um quinto dos parâmetros e dos MACs. [Uma seed só: as seeds 43 e 44 desta rede estão na fila.] Na seed 42, os dois kernels ficam juntos com o Fire (60,20% e 60,11%), enquanto os corpos comuns ficam 1,82 p.p. separados. Já o Bottleneck empata com o corpo comum no 3×3 (+0,40 p.p.) e perde 1,78 p.p. no 2×2.

Para um acelerador Winograd, o Fire é um meio-termo, porque os blocos põem parte da conta em convoluções 1×1, que o acelerador não executa. Com dois por estágio, o Fire 3×3 deixa no acelerador 76,9% dos seus MACs, ou 283M, contra 1800M da rede comum, e põe 81M em convoluções 1×1. O Fire 2×2 deixa 59,7%, e o Bottleneck, de 40 a 64% nas quatro redes. O Fire 3×3 reduz, portanto, o trabalho do acelerador em cerca de seis vezes sem perder acurácia (uma seed), mas exige outra unidade para as convoluções 1×1. É a mesma tensão da Seção IV-F, em escala menor.

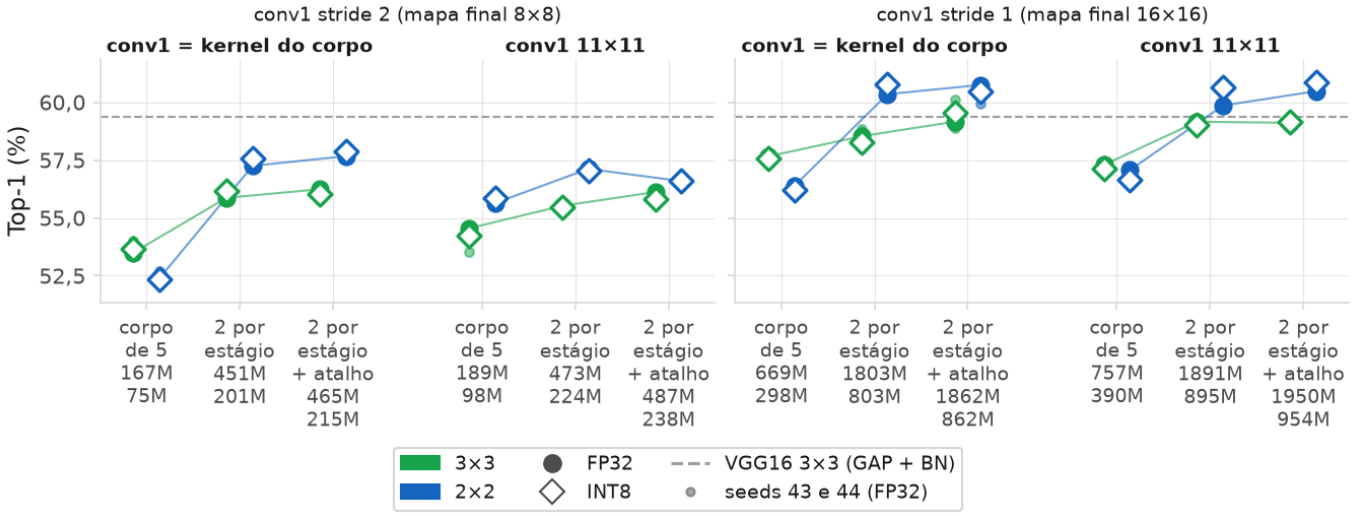


Figura 12. Todas as combinações (AlexNet com GAP e BN, seed 42; variação de um treino: ±0,7 p.p. em FP32). Da esquerda para a direita, em cada grupo, o corpo fica mais profundo. O painel da direita (stride 1, mapa final 16×16) fica acima do da esquerda (stride 2, mapa final 8×8), e o azul (2×2) passa o verde (3×3) quando há dois kernels por estágio. A linha tracejada é a VGG16 3×3. Os pontos pequenos são as seeds 43 e 44, onde houver. Sob cada corpo estão os MACs do 3×3 e, abaixo, os do 2×2.

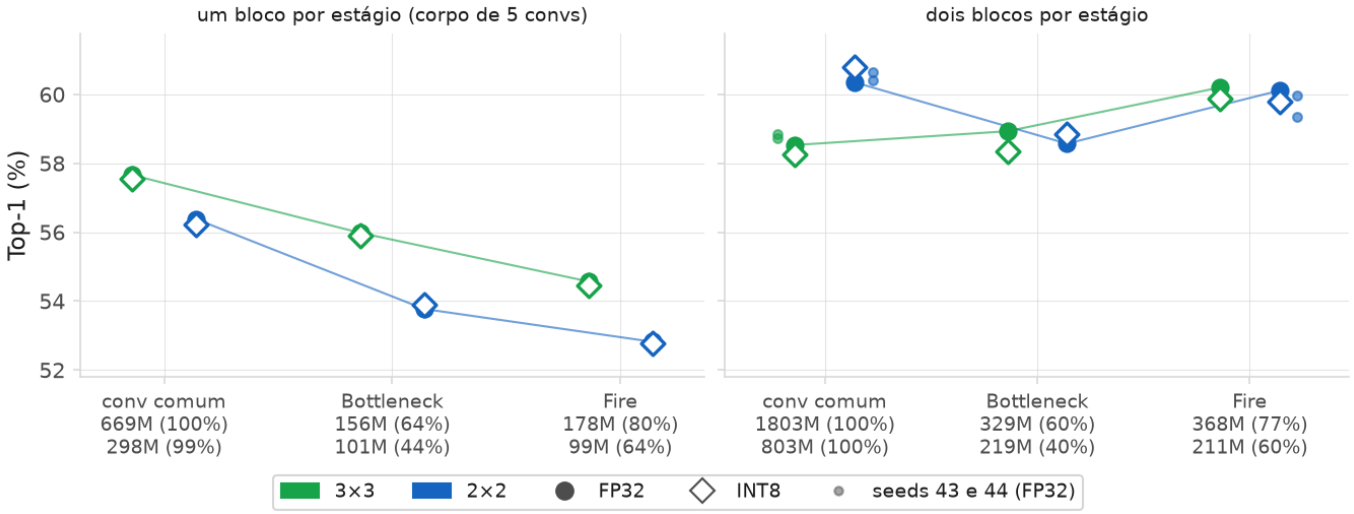


Figura 13. Blocos Bottleneck e Fire no corpo de stride 1 (AlexNet com GAP e BN, seed 42; variação de um treino: ±0,7 p.p. em FP32). À esquerda, um bloco por estágio; à direita, dois. Círculo cheio: FP32; losango vazio: INT8; os pontos pequenos são as seeds 43 e 44, onde houver. Sob cada bloco estão os MACs do 3×3 e, abaixo, os do 2×2; entre parênteses, a parte que um acelerador Winograd executa (a regra da Tabela VII). A Fire 3×3 com um bloco por estágio é a Fire do estudo principal.

J. Quanto a aritmética do acelerador Winograd custa em acurácia?

O custo **depende do tamanho do bloco de saída** (Figura 14). Os MACs das seções anteriores supõem uma conta exata, mas um acelerador Winograd em FPGA faz a conta em ponto fixo, e o erro cresce com o bloco [2]. Nas 16 redes simuladas, a mediana da diferença para o FP32, no conjunto de teste, é de −0,2 p.p. em F(2,3) (de −0,7 a +3,8), −6,3 p.p. em F(4,3) (de −8,6 a +0,5) e −40,1 p.p. em F(6,3) (de −59,8 a −10,5). Os valores mais altos de F(2,3) e F(4,3) são da AlexNet 3×3 de layout original, cuja melhor época FP32 veio cedo: ali o QAT também é treino extra (Seção IV-E).

No F(6,3), porém, não se trata só de uma perda maior. Seis redes profundas (VGG-Style, VGG-13, VGG16, WRN-16-4, WRN-28-2 e ResNet-18) caem ao acaso, com 0,4 a 0,6% de acerto, e a RepVGG-A0 fica em 4,2%, enquanto as AlexNets ficam entre 11 e 28%, e a GoogLeNet, em 48,5%. Um estudo anterior do grupo, com outra receita (AdamW, FP32 de até 100 épocas, 15 épocas de QAT medidas na validação), tinha achado +0,05, −5,1 e −18,9 p.p., e nele a mesma ResNet-18 chegava a 37,5% em F(6,3).

A diferença entre os dois estudos não é falha de treino. Medimos a ResNet-18: cada camada F(6,3), sozinha, erra de forma parecida nas duas receitas (cosseno com a convolução

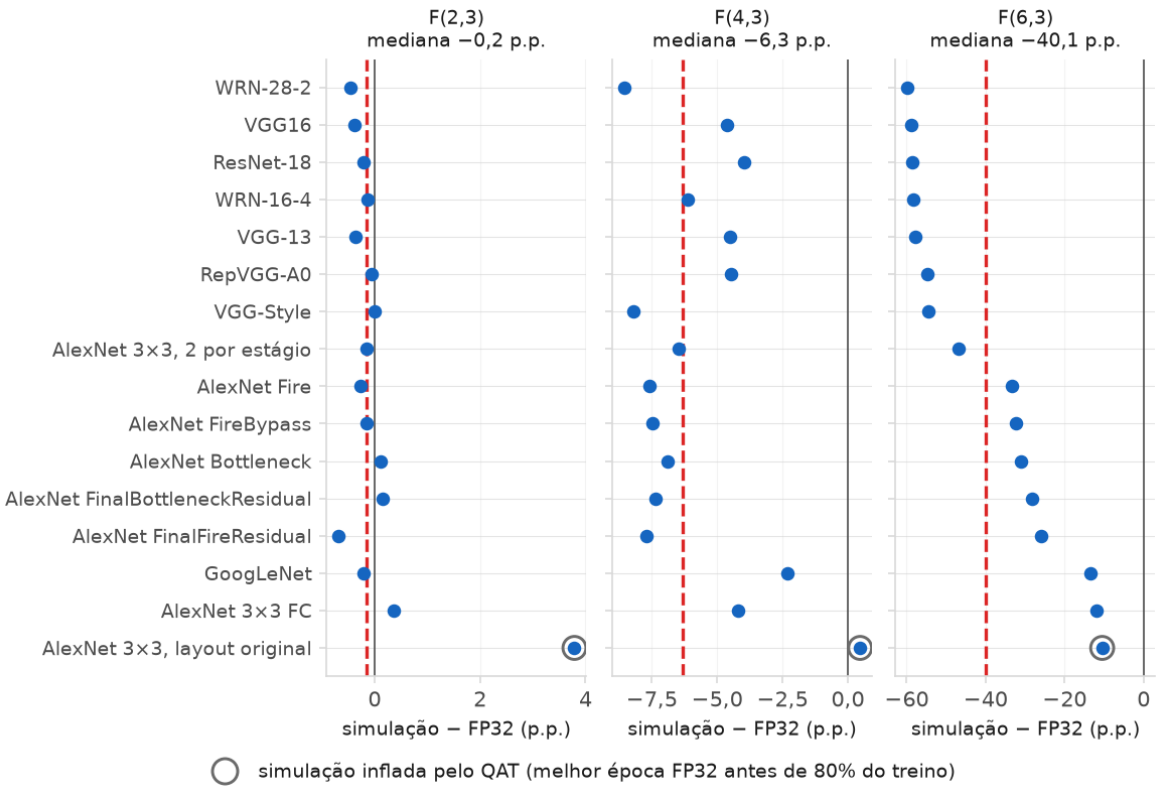


Figura 14. Quanto a aritmética simulada do acelerador Winograd muda a acurácia de cada rede: simulação − FP32 no conjunto de teste (uma seed por rede), um painel por tamanho de bloco. A linha tracejada é a mediana das 16 redes; cada painel tem sua escala. O anel marca a rede cuja melhor época FP32 veio cedo.

FP32 de 0,46 a 0,96 na nova e de 0,75 a 0,96 na anterior). O que muda é o acúmulo do erro, porque os observadores da quantização se calibram nas ativações da própria rede simulada, que já carregam o erro das camadas anteriores. Na receita única, a partir da sétima convolução (13 de 19), eles ficam de 14 a 51 vezes acima do sinal limpo (de 1 a 3,7 vezes na anterior, em todas). Nessas convoluções, só 2% a 57% das entradas diferentes de zero mantêm um código INT8 diferente de zero, e menos de 2% dos blocos transformados passam: quase nada chega ao fim da rede.

A VGG-13 colapsa do mesmo jeito: a partir da sexta convolução (5 de 10), os observadores ficam de 12 a 34 vezes acima do sinal, e só 0,1% a 2,4% dos blocos passam. Já a GoogLeNet, com a mesma receita, não acumula o erro: os observadores ficam no máximo 4,3 vezes acima do sinal, e pelo menos 29% dos blocos passam em todas as convoluções. No início do QAT, as três estão no acaso em F(6,3), mas só a GoogLeNet chega a 48,5%. O que decide, portanto, é **se o sinal atravessa a rede**: sem sinal, o QAT não tem de onde partir. Não medimos por que o erro se acumula em umas redes e não em outras.

V. Discussão

A. *Como desenhar uma rede para Winograd*

Os resultados apontam para algumas escolhas:

- Acerte primeiro a geometria e o treino: use stride baixo na conv1 para imagens pequenas, BN e, na AlexNet, GAP. Esses fatores pesam tanto ou mais que o kernel. O stride 1 é o mais caro, pois quadruplica os MACs, e cabe a cada projeto decidir se vale esse custo.
- Não aumente o kernel para ganhar acurácia: com GAP e BN, gastar a mais em largura ou em uma segunda convolução por estágio rende mais.
- Use a conv1 11×11 com cuidado. Ela ajuda um corpo de 5 convoluções, mas não acrescenta nada com dois por estágio e tira de 5 a 23 p.p. da parcela Winograd (10 p.p. no 2×2 com dois por estágio).
- Evite redes dominadas por convoluções *depthwise* ou 1×1 se o alvo é Winograd. Se a meta também é cortar custo, o Fire com dois blocos por estágio usa de um quarto a um quinto dos MACs e ainda deixa de 60 a 77% deles no acelerador; em acurácia, mantém a do corpo 3×3 (uma seed) e custa 0,7 p.p. no 2×2 (3 seeds; Seção IV-I).
- Não trate a quantização como obstáculo, mas confira se o FP32 convergiu antes do QAT, para não confundir treino extra com efeito da quantização.

B. *2×2 ou 3×3?*

A escolha **depende do hardware**. Em acurácia, com dois por estágio, o 2×2 fica à frente e faz cerca de metade dos MACs diretos, ou cerca de 1,8 vez menos com Winograd

(Seção IV-F). Esse kernel, porém, precisa de um acelerador preparado para ele.

A aritmética do acelerador também pesa (Seção IV-J). Os resultados de F(m,3) vêm da simulação, não do hardware, e o kernel 2×2 fica fora dessas transformações, então não há medida equivalente para F(m,2). Assim, um acelerador com F(2,3) e redes 3×3 é **a opção sem custo de acurácia medido**. Com F(4,3), a perda é de cerca de 6 p.p. (no pior caso, 8,6), e com F(6,3) o resultado depende da rede e, nas profundas, é inutilizável sem recalibrar as escalas.

C. O que muda em relação ao trabalho anterior

Sob a mesma receita, **boa parte das diferenças entre redes de compensação desaparece**. No relatório anterior [7], um atalho residual em um bloco Fire [33] parecia ganhar mais de 6 p.p., mas os treinos seguiam protocolos diferentes, inclusive no número de épocas. Agora, Fire fica em 54,57% e FireBypass em 55,19%, uma diferença de 0,62 p.p., dentro da variação. As redes Fire, FireBypass e as combinações com Fire e Bottleneck sem *depthwise* ficam a menos de 1,5 p.p. umas das outras (de 54,40% a 55,83%, seed 42). O estudo de combinações aponta na mesma direção: o atalho não mostra ganho mensurável. Isso não invalida esses blocos, que continuam baratos, mas muda o motivo para usá-los.

As demais fases do projeto entram só como contexto, porque usaram protocolos diferentes. Medições anteriores em GPU indicaram que a aceleração Winograd depende de convoluções densas de stride 1. Em detecção, a compensação estrutural se transferiu só em parte. Em atenção local, o tamanho da janela se comportou como o tamanho do kernel [7].

D. Limitações

A principal limitação é **o número de seeds**: a maioria das comparações usa uma só. As duas melhores redes só com 2×2 foram repetidas com as seeds 43 e 44 e continuam acima da VGG16, mas as outras 22 combinações têm só a seed 42, entre elas a com conv1 11×11 que empata com essas duas. Os blocos Bottleneck e Fire também têm só a seed 42, exceto o Fire 2×2 com dois blocos por estágio, repetido com as seeds 43 e 44.

O alcance da receita e dos controles também é limitado. A receita única não é a melhor para cada rede, e uma receita pode mudar a ordem entre arquiteturas [8], [9]; a taxa 0,02, por exemplo, pode melhorar as três redes testadas. O controle de capacidade e os testes de taxa e de CIFAR-100 usam GAP e BN, e o custo do 2×2 com FC na AlexNet não foi explicado nem repetido. Os dados têm 64 px e 32 px, e as imagens de 224 px são apenas ampliações. Além disso, os três treinos da MobileNetV2 pré-treinada ficaram abaixo da nossa guarda de conversão, e a causa não foi investigada.

Do lado do hardware, **nada foi medido em um acelerador real**. Os resultados F(m,3) têm uma seed por rede e vêm da simulação, e as escalas recalibradas, que poderiam evitar o colapso em F(6,3), não foram testadas. A parcela de MACs e a estimativa com Winograd usam só a geometria das camadas, e não medimos a vazão. Também não comparamos latência, porque os treinos rodaram em máquinas diferentes. Essa diferença faz parte da variação entre execuções e também afeta a seed 42 do estudo de combinações: as duas melhores redes foram treinadas em GPUs diferentes.

VI. Conclusão

Restringir o kernel **custa pouco quando o resto da rede está bem escolhido**. Com GAP e BN, trocar os kernels originais da AlexNet por 3×3 não custa nada, e a rede inteira em 2×2 perde só 0,5 p.p. Nessa configuração, o custo do 2×2 em relação ao 3×3 vem da capacidade: com os mesmos MACs e parâmetros, os dois empatam. Com FC, porém, o custo na AlexNet chega a 6,5 p.p., e na VGG16 os dois empatam. Por isso, o custo do kernel deve ser lido junto com o classificador e a família da rede.

A geometria e o BN costumam pesar mais que o kernel, e a quantização INT8 quase não pesa. Duas convoluções pequenas por estágio recuperam a perda e, nos pares testados, o 2×2 fica à frente do 3×3 com cerca de metade dos MACs. Com três seeds, essa vantagem de 1 a 2 p.p. se confirma sem atalho, mas não com atalho. Na aritmética do acelerador, o tamanho do bloco de saída custa mais acurácia do que a troca de kernel: F(2,3) quase não custa, e F(6,3) inutiliza a maioria das redes profundas. Com GAP e BN, o custo e a ordem dos kernels se repetem com outras taxas de aprendizado e no CIFAR-100.

Para quem projeta redes para aceleradores Winograd, a mensagem é simples: **não é preciso manter kernels grandes no corpo da rede para ter boa acurácia**, desde que a geometria e o classificador sejam escolhidos para isso.

VII. Trabalhos Futuros

Queremos:

- fazer o controle de capacidade com classificador FC, para explicar o custo do 2×2 na AlexNet;
- reavaliar todas as redes no mesmo tipo de máquina, para comparar latência;
- medir a vazão em um acelerador real;
- investigar a MobileNetV2 pré-treinada em INT8;
- repetir as comparações principais em imagens de resolução maior.

Agradecimentos

Experimentos utilizaram recursos da infraestrutura PCAD (http://pcad.inf.ufrgs.br).

Referências

[1] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Error analysis and improving the accuracy of Winograd convolution for deep neural networks," *ACM Transactions on Mathematical Software*, vol. 46, no. 4, Art. 37, 2020.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.
[6] X. Ding, X. Zhang, Y. Zhou, J. Han, G. Ding, and J. Sun, "Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.
[7] R. S. de Souza and M. G. da Silva, "Restrição do tamanho de kernel em CNNs para aceleração Winograd," relatório de pesquisa, Instituto de Informática, UFRGS, 2026.
[8] I. Bello *et al.*, "Revisiting ResNets: Improved training and scaling strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
[9] R. Wightman, H. Touvron, and H. Jégou, "ResNet strikes back: An improved training procedure in timm," *arXiv preprint arXiv:2110.00476*, 2021.
[10] I. Radosavovic, J. Johnson, S. Xie, W.-Y. Lo, and P. Dollár, "On network design spaces for visual recognition," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
[11] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," Stanford CS231N Report, 2015.
[12] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "A downsampled variant of ImageNet as an alternative to the CIFAR datasets," *arXiv preprint arXiv:1707.08819*, 2017.
[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
[14] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, 2010.
[15] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.
[16] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of tricks for image classification with convolutional neural networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.
[17] P. Goyal *et al.*, "Accurate, large minibatch SGD: Training ImageNet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.
[18] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
[19] M. Li, E. Yumer, and D. Ramanan, "Budgeted training: Rethinking deep neural network training under resource constraints," in *International Conference on Learning Representations (ICLR)*, 2020.
[20] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123.
[21] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015.
[22] P. Micikevicius *et al.*, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018.
[23] S. Wu, G. Wang, P. Tang, F. Chen, and L. Shi, "Convolution with even-sized kernels and symmetric padding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
[24] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
[25] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.
[26] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.
[27] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
[28] X. Bouthillier *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Machine Learning and Systems (MLSys)*, 2021.
[29] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
[30] S. Hooker, A. Courville, G. Clark, Y. Dauphin, and A. Frome, "What do compressed deep neural networks forget?" *arXiv preprint arXiv:1911.05248*, 2019.
[31] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1–2, pp. 81–93, 1938.
[32] H. Li *et al.*, "Rethinking the hyperparameters for fine-tuning," in *International Conference on Learning Representations (ICLR)*, 2020.
[33] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," *arXiv preprint arXiv:1602.07360*, 2016.

APÊNDICE

A. Figuras de apoio

As figuras a seguir trazem o detalhe por trás do texto: o efeito do stride da conv1, as imagens trocadas entre FP32 e INT8, a transferência para o CIFAR-100, a VGG16 com um kernel por bloco e as curvas de treino e de QAT, que mostram que as redes param de melhorar antes do fim das épocas.

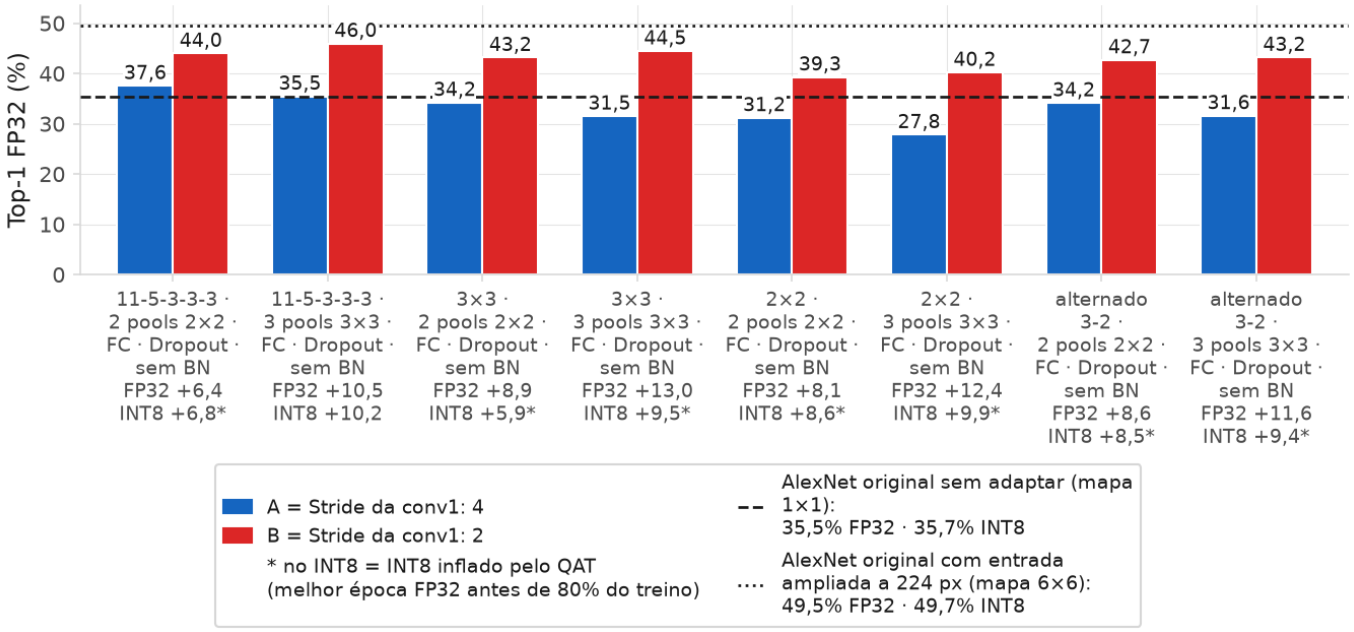


Figura 15. O stride da conv1 pesa mais que o kernel. Em cada grupo, duas redes iguais exceto pelo stride: 4 (azul) ou 2 (vermelho). O stride 2 ganha nos oito pares. Barras: FP32 no teste, seed 42 (média das seeds onde houver). Sob cada grupo: B − A em FP32 e em INT8 (asterisco: INT8 inflado pelo QAT). Linhas horizontais: a AlexNet sem adaptação e com a imagem ampliada para 224 px.

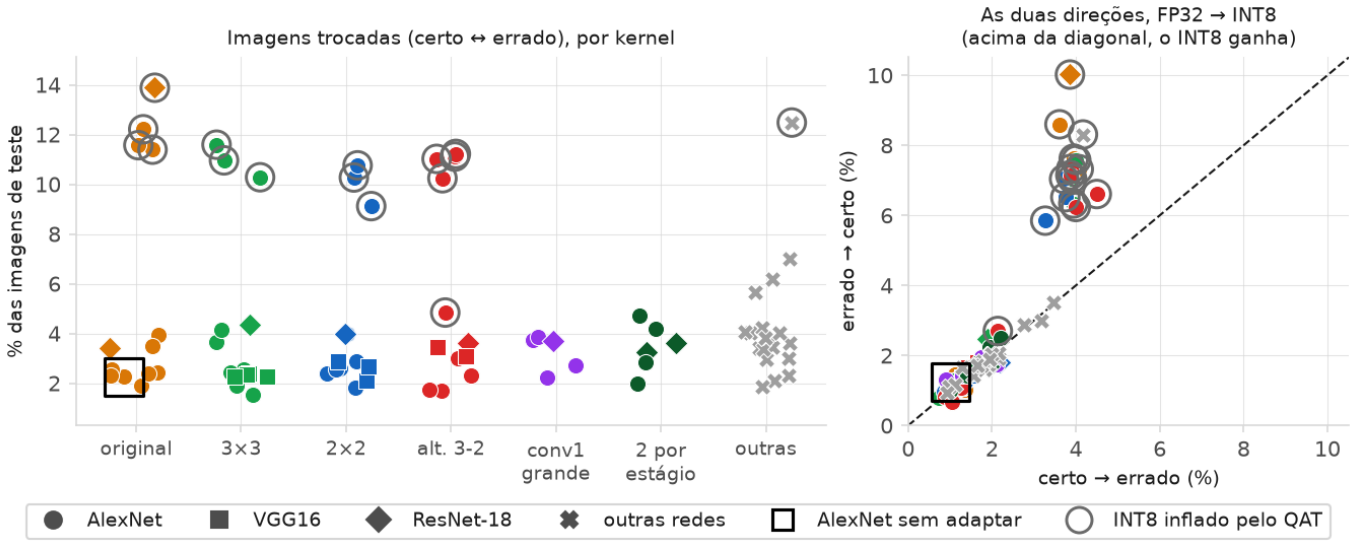


Figura 16. Imagens de teste que mudam entre certo e errado do FP32 para o INT8 (seed 42). À direita, sobre a diagonal, o saldo é zero. Só as redes com anel ficam claramente acima dela. O quadrado marca a AlexNet sem adaptação; os grupos são os da Figura 8.

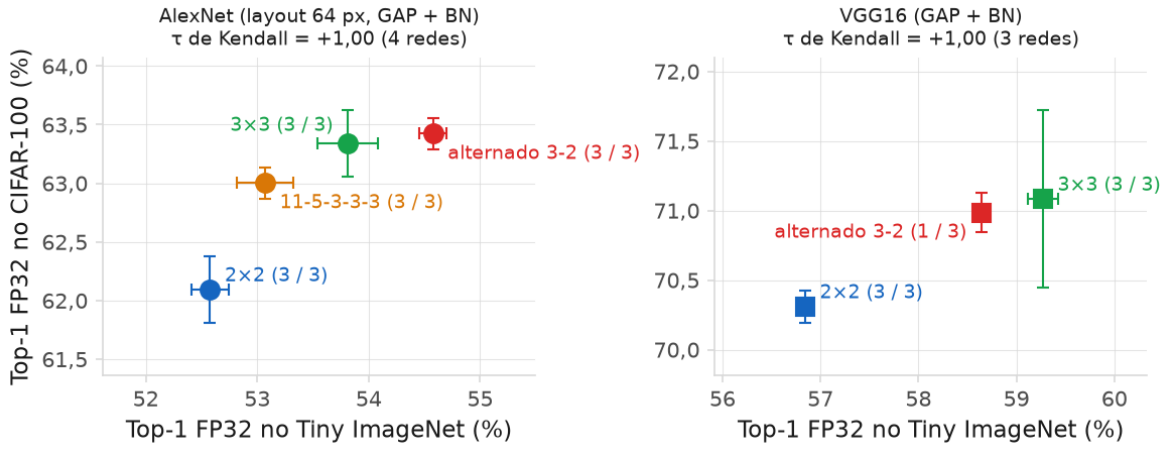


Figura 17. A ordem dos kernels no Tiny ImageNet se repete no CIFAR-100, com GAP e BN. Cada ponto é a média das seeds, e as barras mostram um desvio-padrão entre seeds. Entre parênteses, quantas seeds no Tiny ImageNet e no CIFAR-100.

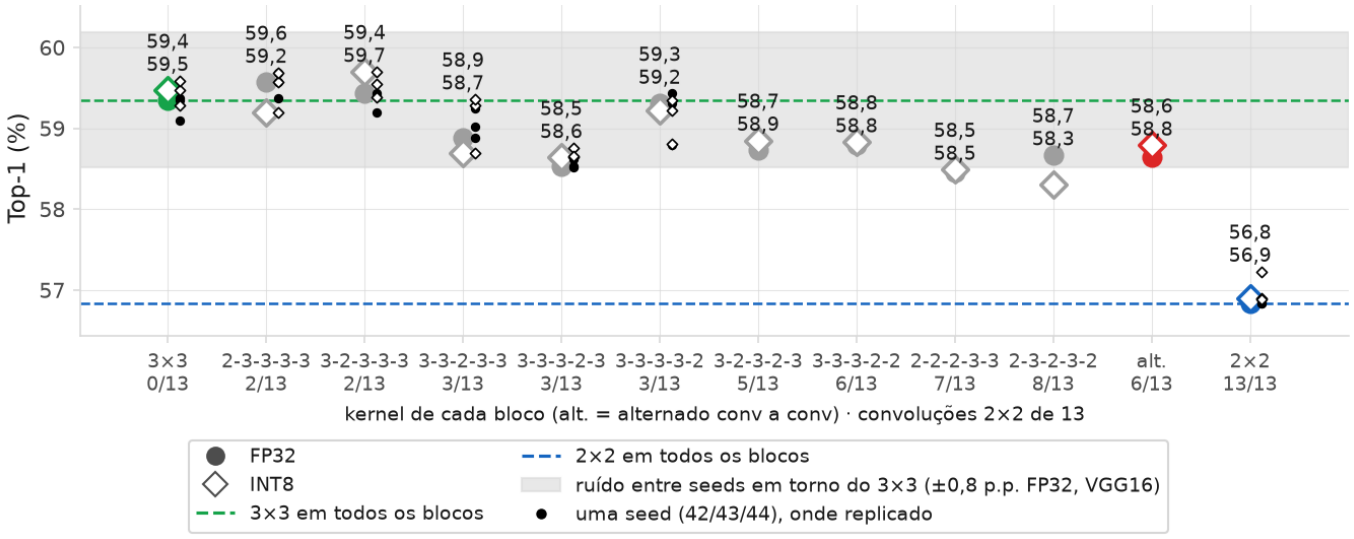


Figura 18. VGG16 com um kernel por bloco (GAP e BN; pontos: seed 42 e, onde houver, as outras seeds). Os padrões leem-se bloco a bloco: 3-3-3-2-3 tem 2×2 só no bloco 4; abaixo, quantas das 13 convoluções são 2×2. A faixa cinza é a variação de um treino em torno do 3×3; a faixa para médias de 3 seeds é menor (Tabela IV).

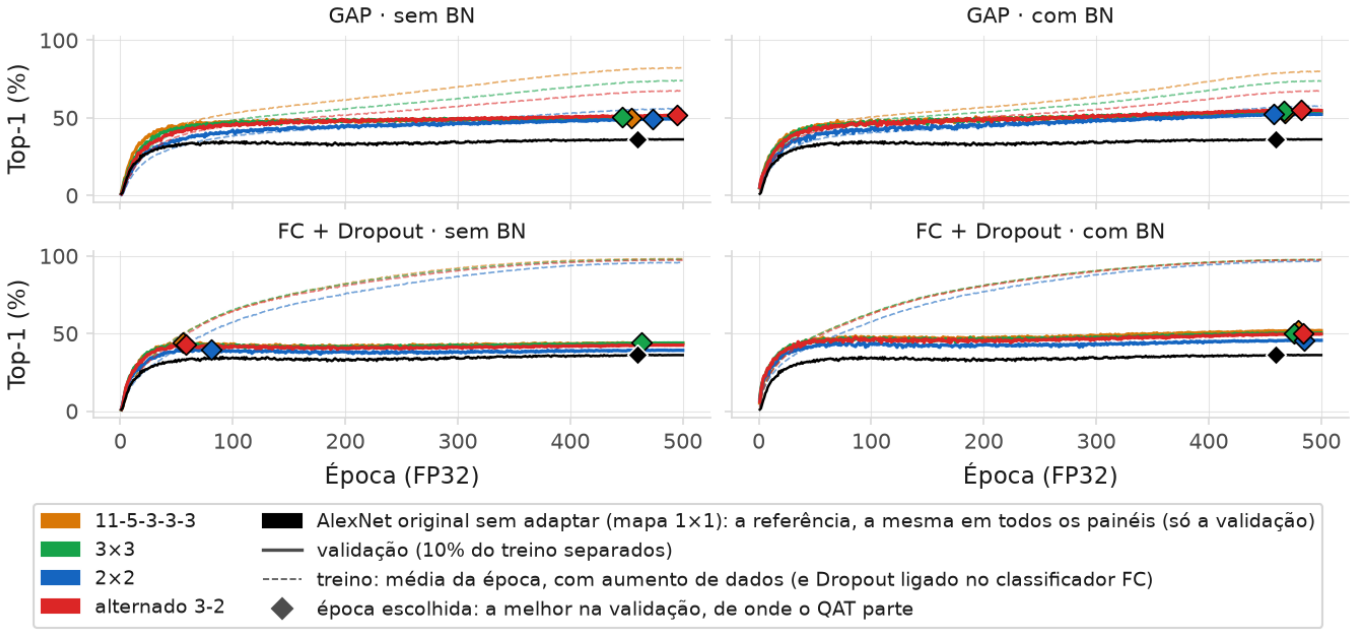


Figura 19. Curvas de treino em FP32 (AlexNet de layout 64 px, seed 42), medidas nos 10% do treino separados para validação. As redes com FC chegam perto de 100% no treino, enquanto a validação fica parada. Nas redes com FC sem BN, a melhor época costuma vir cedo: são os casos marcados no texto.

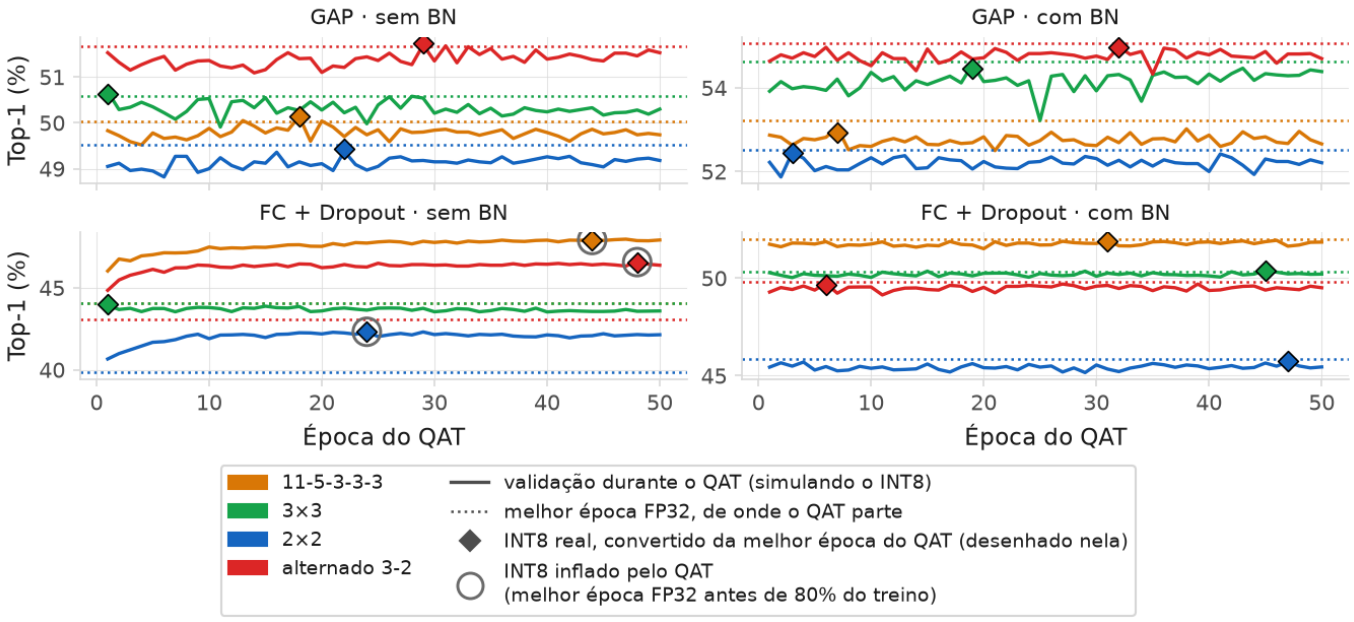


Figura 20. Curvas do QAT nas mesmas redes. O pontilhado é o FP32 de onde o QAT parte, e o losango é a época convertida para INT8. Só as redes com anel sobem claramente acima do ponto de partida: é o treino extra descrito na Seção IV-E.